Deep Learning for Computer Vision (Discriminative way)

Andrii Liubonko

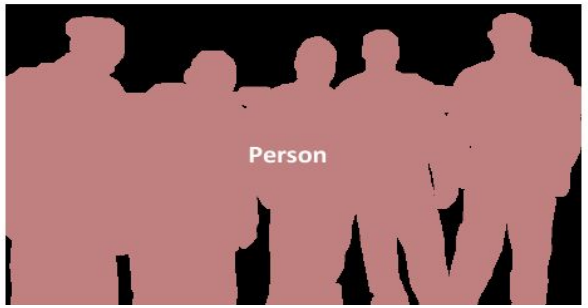
2025

Content

- **Intro to Object Detection**
  - Datasets
  - Metrics
- Two-stage detectors [RCNN Family]
- One-stage detectors
  - YOLO
  - Single Shot Detector [SSD]
- Transformer based detection [DETR]
- Open-World

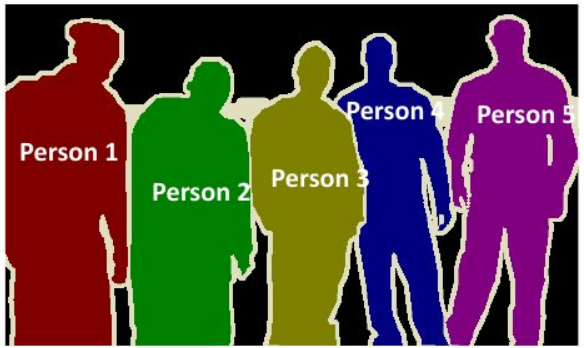
Visual Perception Problems

Classification + Localization



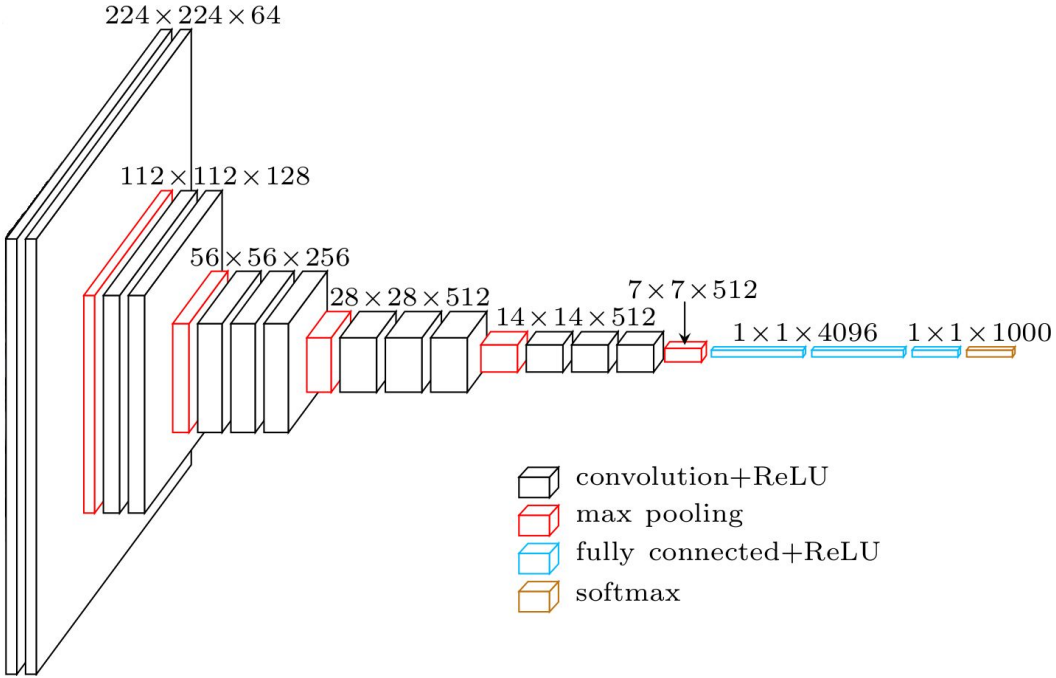
Semantic Segmentation

Object Detection



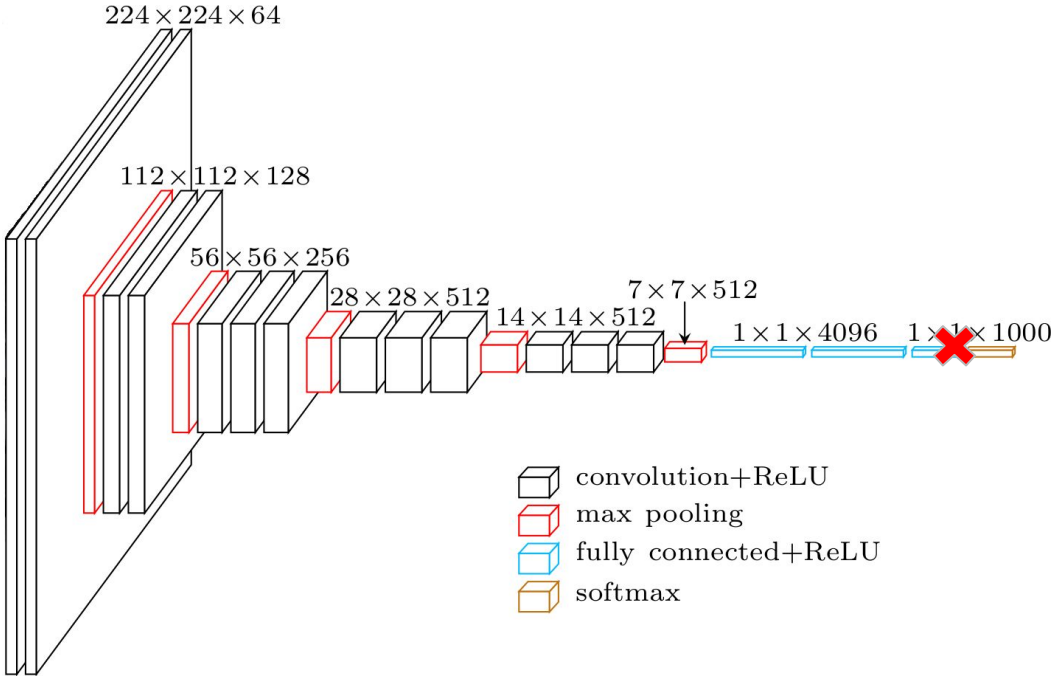
Instance Segmentation

Image Classification



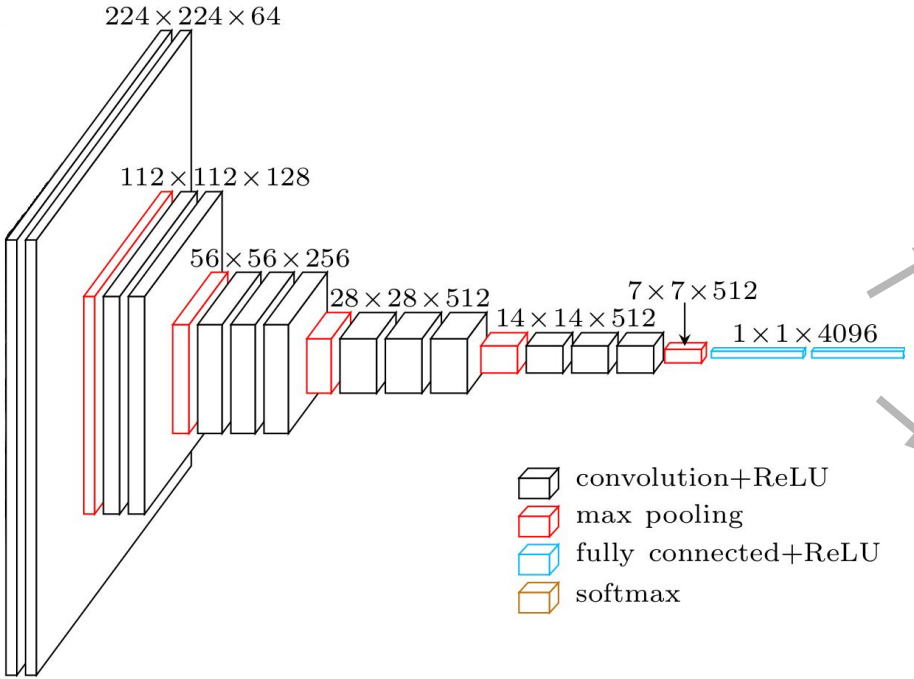
dog: 0.98
cow: 0.01
....

Image Classification -> Classification + Localization



dog: 0.98
cow: 0.01
....

Image Classification -> Classification + Localization



4096x1000

FC

[Score]
dog: 0.98
cow: 0.01
....

Softmax Loss

+

4096x4

FC

α · L2 Loss

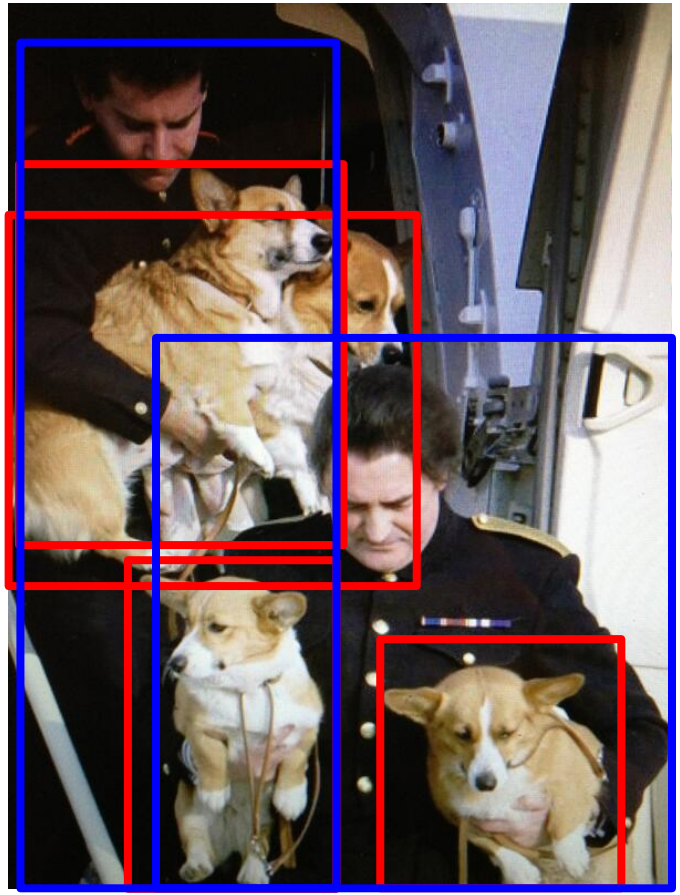
[BB]:
x,y,h,w

Multihead Network
(Multitask Loss)

Detection

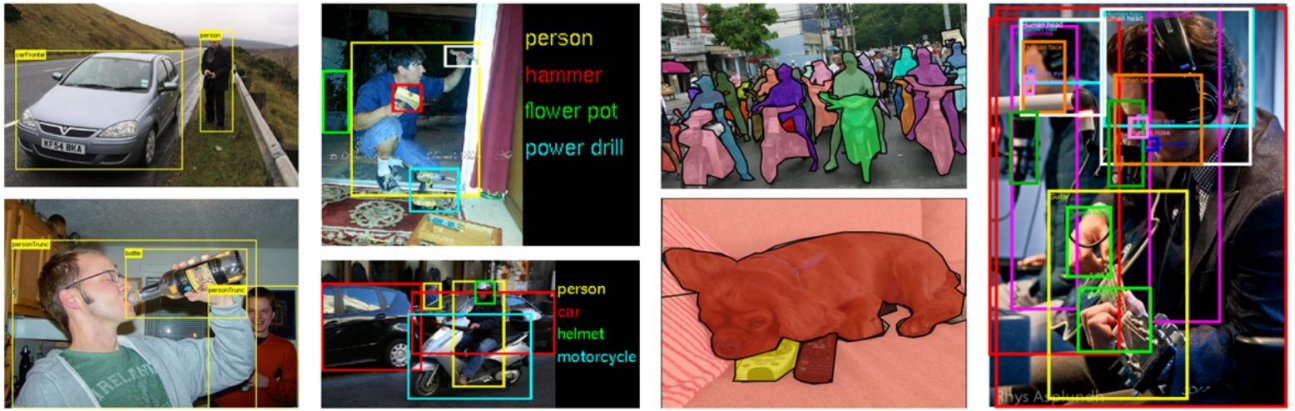
Object Detection

dog human



Object Detection [Datasets]

Dataset	train		validation		trainval		test	
	images	objects	images	objects	images	objects	images	objects
VOC-2007	2,501	6,301	2,510	6,307	5,011	12,608	4,952	14,976
VOC-2012	5,717	13,609	5,823	13,841	11,540	27,450	10,991	-
ILSVRC-2014	456,567	478,807	20,121	55,502	476,688	534,309	40,152	-
ILSVRC-2017	456,567	478,807	20,121	55,502	476,688	534,309	65,500	-
MS-COCO-2015	82,783	604,907	40,504	291,875	123,287	896,782	81,434	-
MS-COCO-2018	118,287	860,001	5,000	36,781	123,287	896,782	40,670	-
OID-2018	1,743,042	14,610,229	41,620	204,621	1,784,662	14,814,850	125,436	625,282



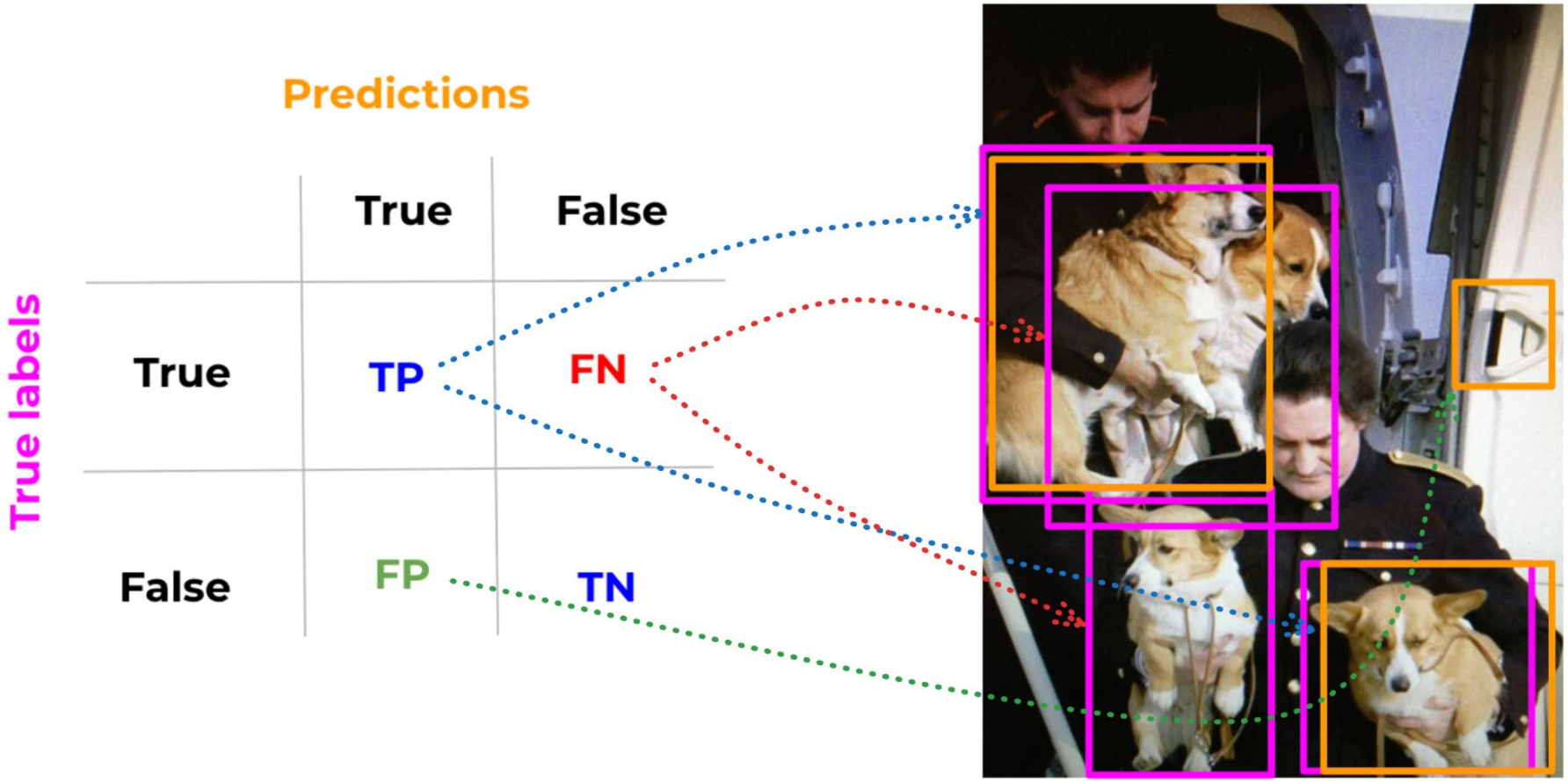
arXiv:1904.01569

VOC ILSVRC MS-COCO OID

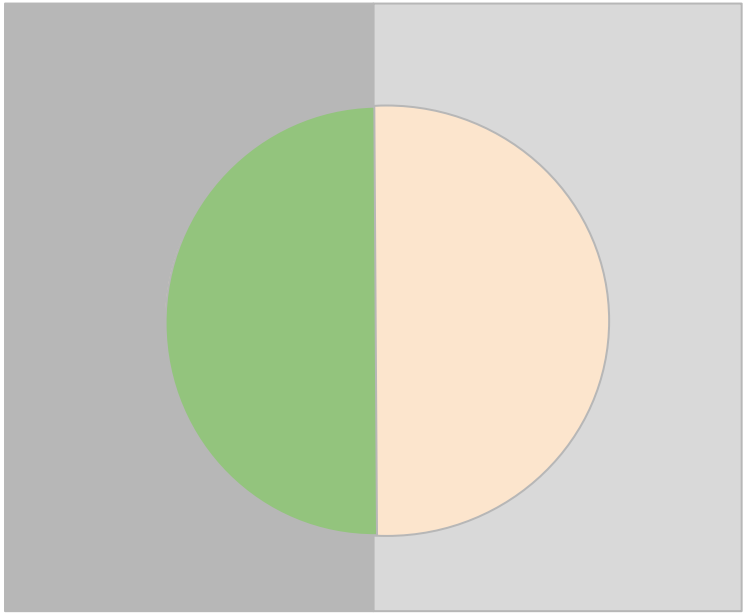
Metrics

Predictions

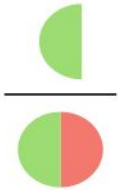
True labels	True	False
True	TP	FN
False	FP	TN


Predictions
True
False
True labels
True
False
TP
FN
FP
TN

Object Detection [Metrics]



How many selected items are relevant?

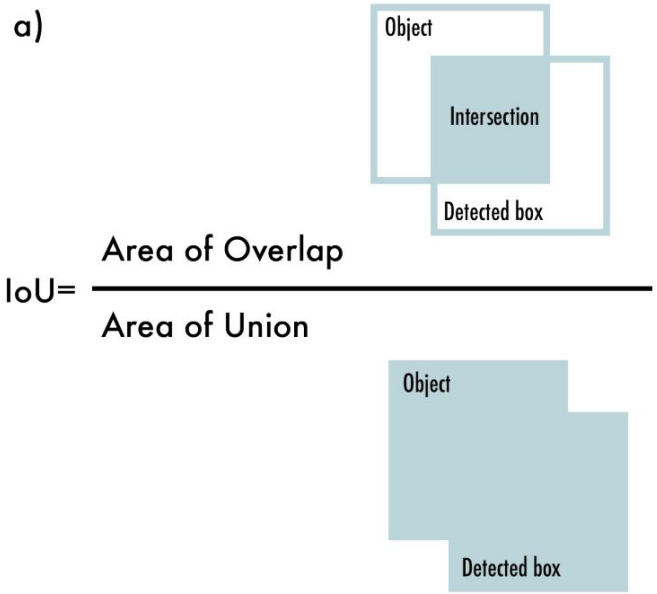


How many relevant items are selected?

$$\text{Precision} = \frac{tp}{tp + fp} \qquad \text{Recall} = \frac{tp}{tp + fn}$$

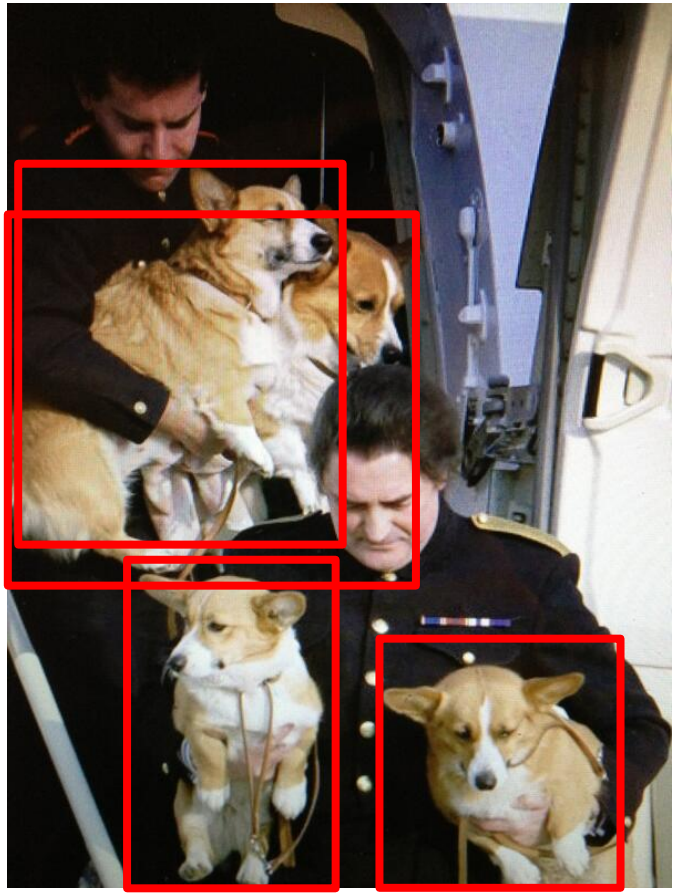
https://en.wikipedia.org/wiki/Precision_and_recall

Object Detection [Metrics]



The detection is *true positive (TP)* if ***IoU ≥ threshold @0.5***

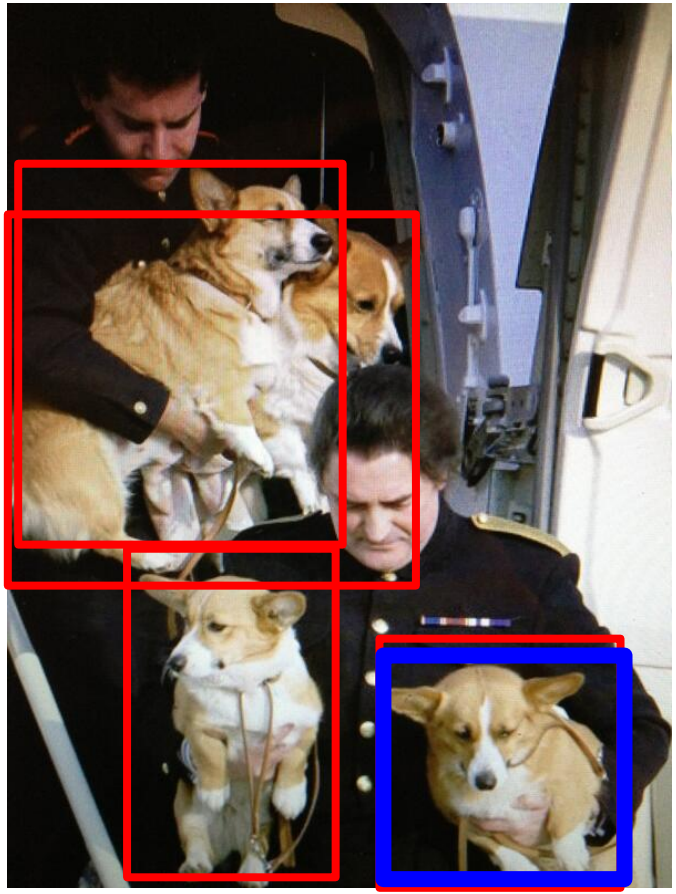
Object Detection [Metrics]



GT

P

Object Detection [Metrics]

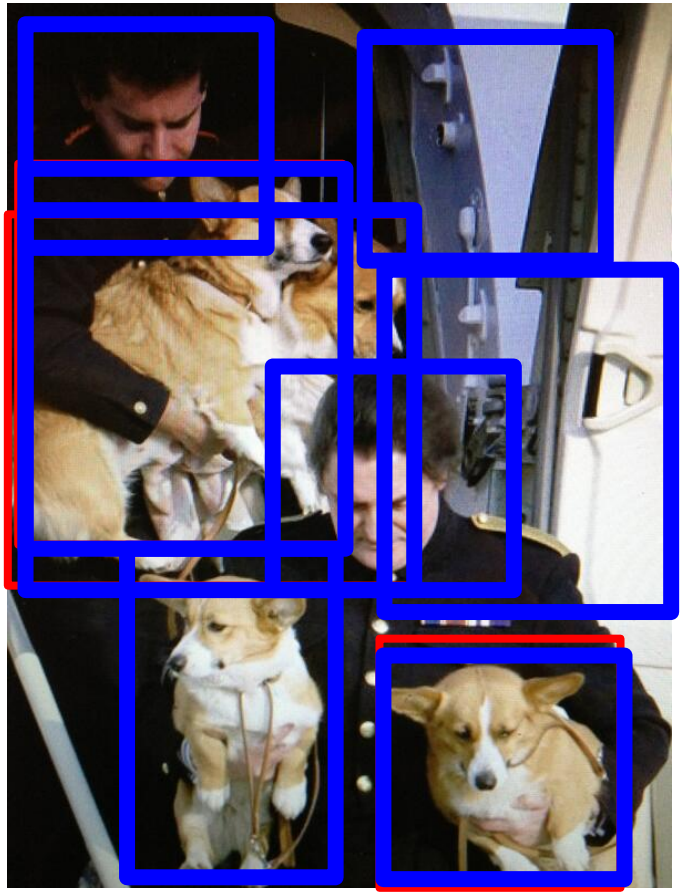


GT

P

Precision ?
Recall ?

Object Detection [Metrics]



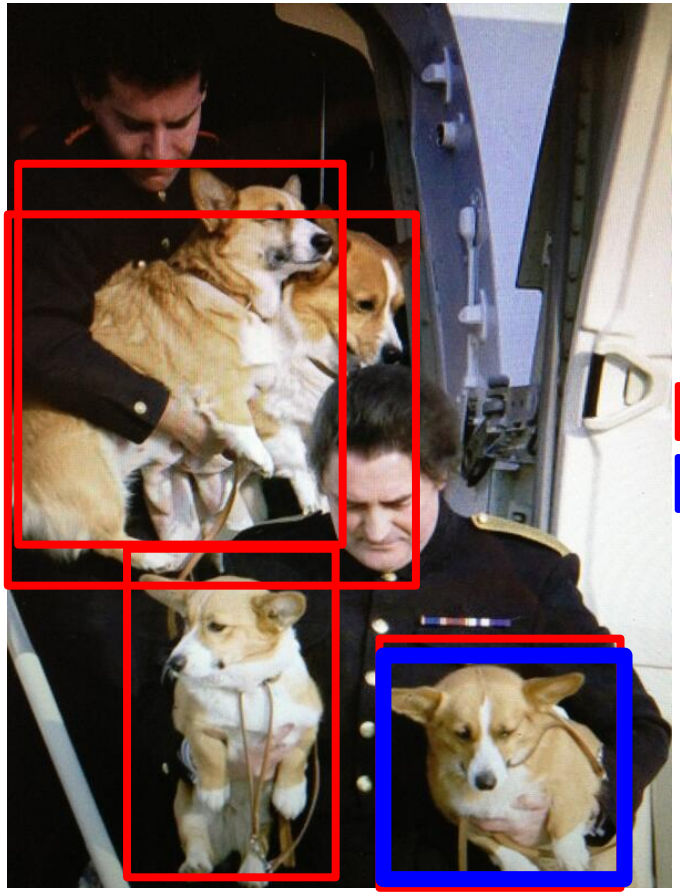
GT

P

Precision ?

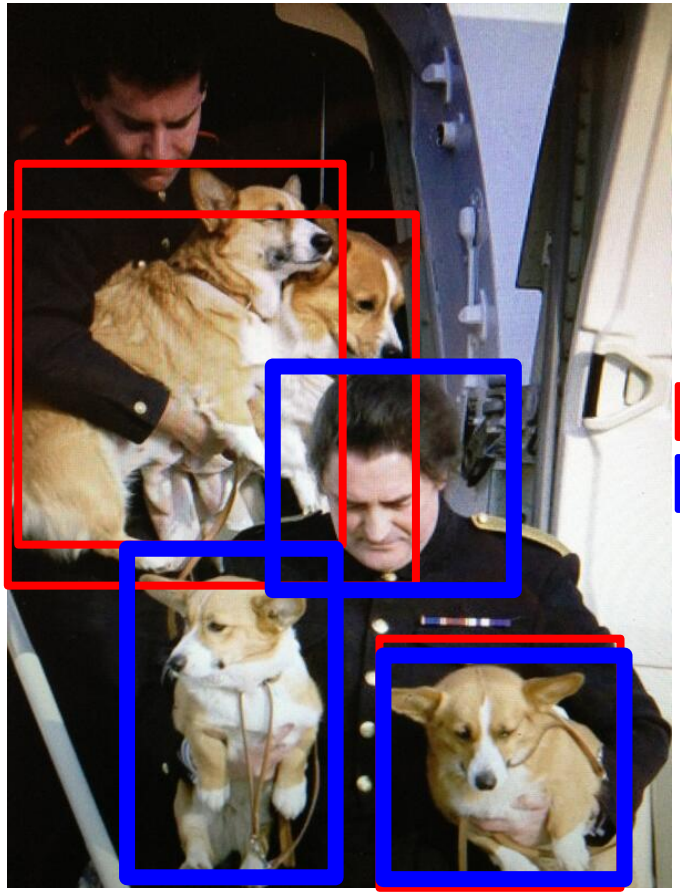
Recall ?

Object Detection



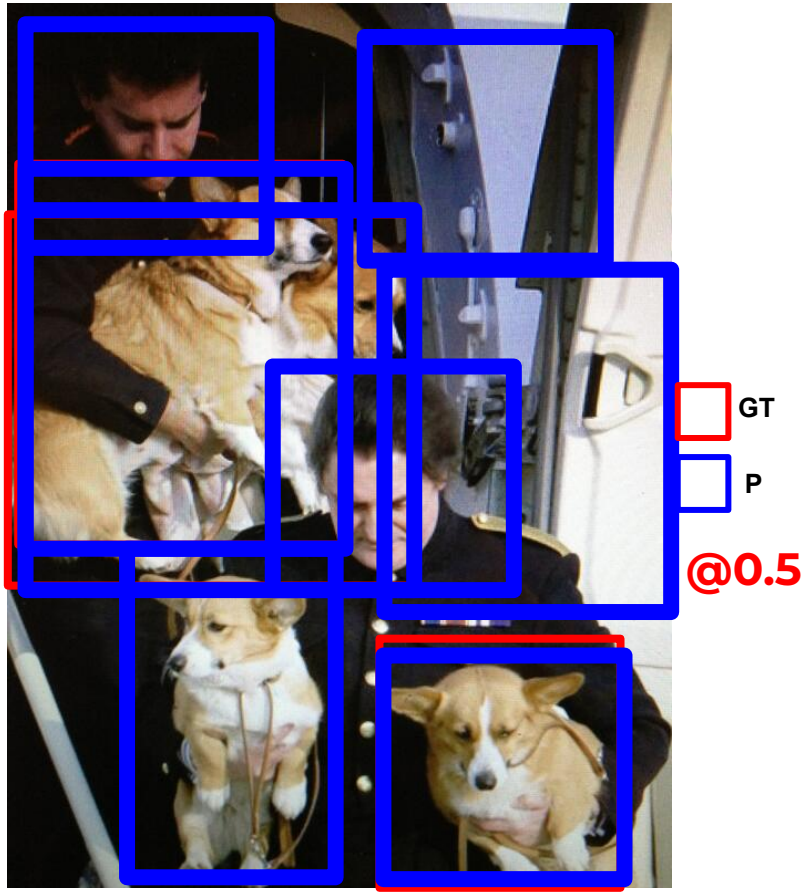
score	correct	Precision	Recall
1.0	True		

Object Detection



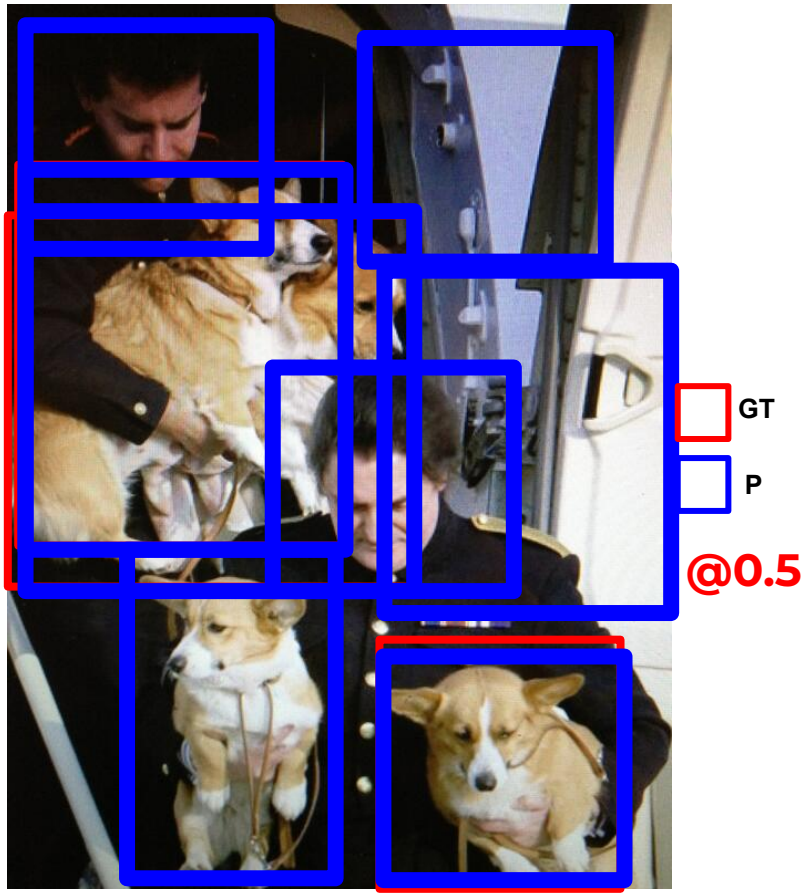
score	correct	Precision	Recall
1.0	True		
0.98	True		
0.97	False		

Object Detection



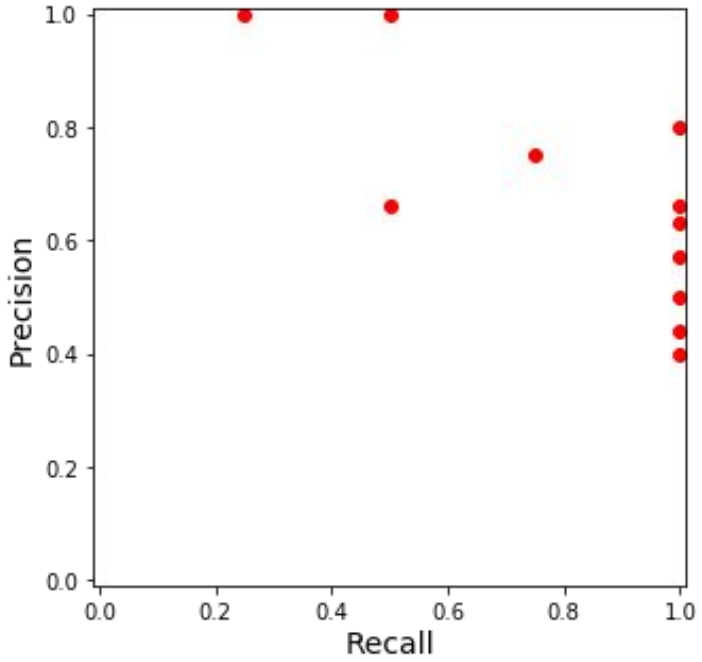
score	correct	Precision	Recall
1.0	True		
0.98	True		
0.97	False		
0.81	True		
0.77	True		
0.67	False		
0.53	True		
0.49	False		
0.33	False		
0.22	False		
0.11	False		

Object Detection



score	correct	Precision	Recall
1.0	True	1.00	0.25
0.98	True	1.00	0.5
0.97	False	0.66	0.5
0.81	True	0.75	0.75
0.77	True	0.80	1.00
0.67	False	0.66	1.00
0.53	True	0.63	1.00
0.49	False	0.57	1.00
0.33	False	0.50	1.00
0.28	False	0.44	1.00
0.14	False	0.4	1.00

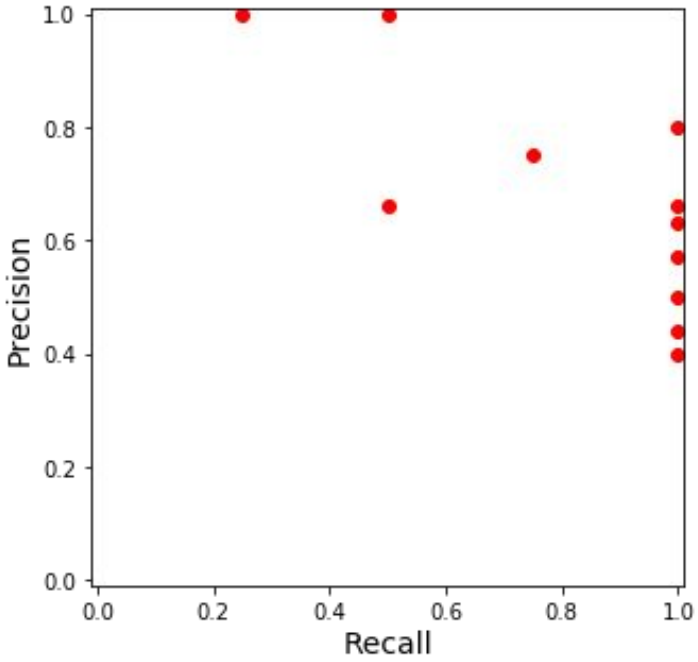
Object Detection

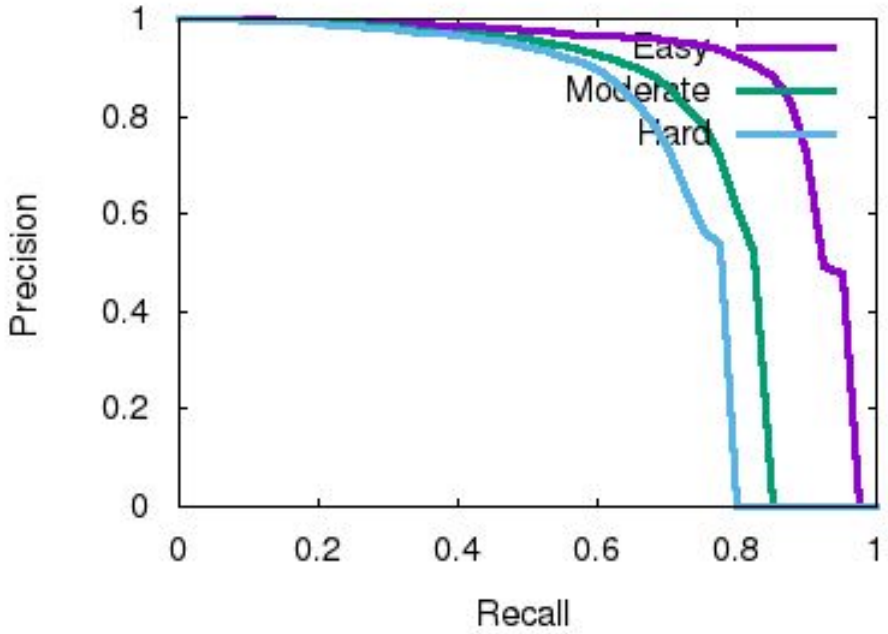


score	correct	Precision	Recall
1.0	True	1.00	0.25
0.98	True	1.00	0.5
0.97	False	0.66	0.5
0.81	True	0.75	0.75
0.77	True	0.80	1.00
0.67	False	0.66	1.00
0.53	True	0.63	1.00
0.49	False	0.57	1.00
0.33	False	0.50	1.00
0.28	False	0.44	1.00
0.14	False	0.4	1.00

@0.5

Object Detection [Metrics]





Object Detection [Metrics]

COCO metrics

1. For each category, calculate the precision-recall curve by varying the confidence threshold of the model's predictions.

2. Compute each category's average precision (AP) using 101-recall thresholds.

3. Calculate AP at different Intersection over Union (IoU) thresholds, typically from 0.5 to 0.95 with a step size of 0.05. A higher IoU threshold requires a more accurate prediction to be considered a true positive.

4. For each IoU threshold, take the mean of the APs across all 80 categories.

5. Finally, compute the overall AP by averaging the AP values calculated at each IoU threshold.

Object Detection [Metrics]

Average Precision (AP):

- AP % AP at IoU=.50:.05:.95 **(primary challenge metric)**
- $AP^{IoU=.50}$ % AP at IoU=.50 (PASCAL VOC metric)
- $AP^{IoU=.75}$ % AP at IoU=.75 (strict metric)

AP Across Scales:

- AP^{small} % AP for small objects: area < 32^2
- AP^{medium} % AP for medium objects: 32^2 < area < 96^2
- AP^{large} % AP for large objects: area > 96^2

Average Recall (AR):

- $AR^{max=1}$ % AR given 1 detection per image
- $AR^{max=10}$ % AR given 10 detections per image
- $AR^{max=100}$ % AR given 100 detections per image

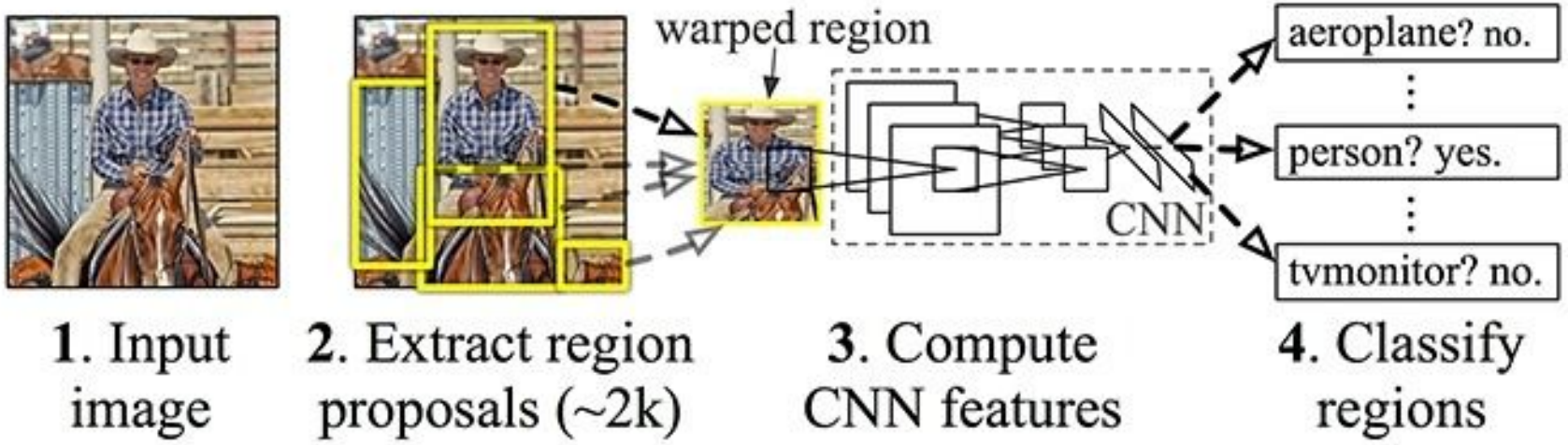
AR Across Scales:

- AR^{small} % AR for small objects: area < 32^2
- AR^{medium} % AR for medium objects: 32^2 < area < 96^2
- AR^{large} % AR for large objects: area > 96^2

Content

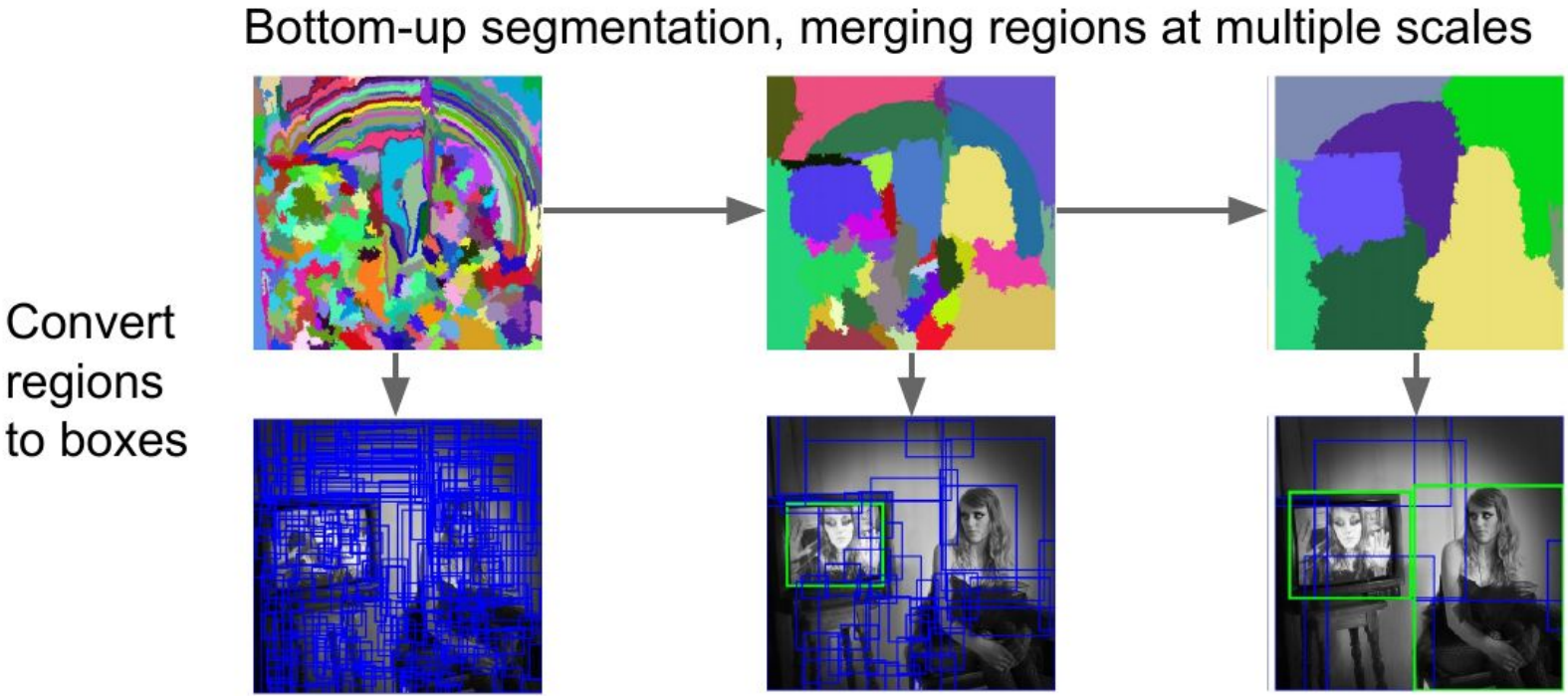
- Intro to Object Detection
  - Datasets
  - Metrics
- **Two-stage detectors [RCNN Family]**
- One-stage detectors
  - YOLO
  - Single Shot Detector [SSD]
- Proposal-free detection
- Summary

R-CNN: Regions with CNN features



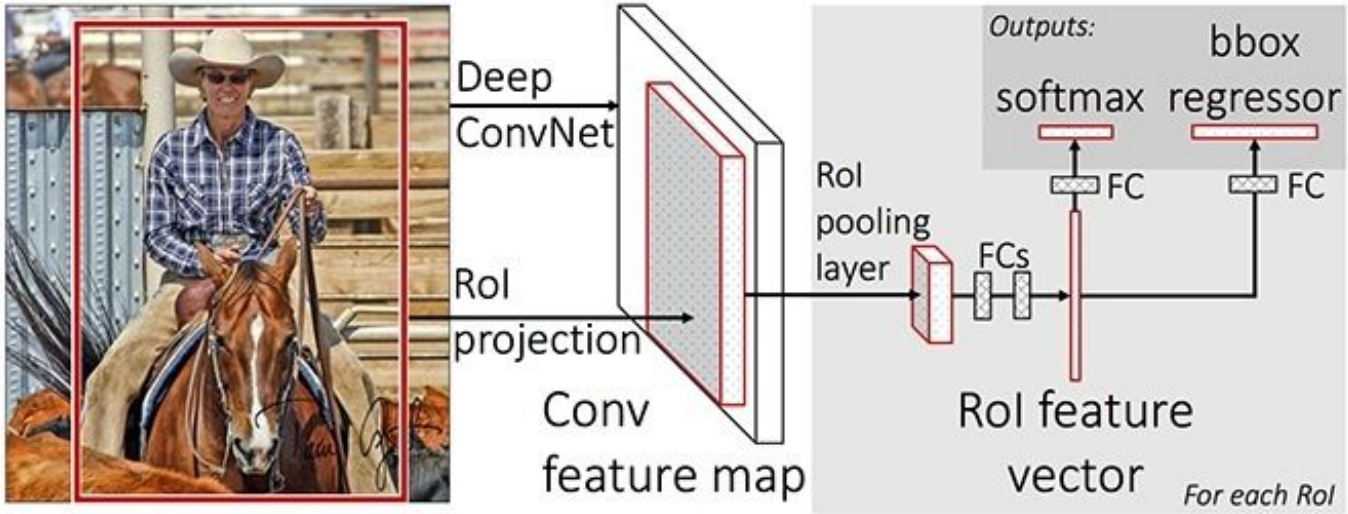
- Extract possible objects using a region proposal method (the most popular one being Selective Search).
- Extract features from each region using a CNN.
- Classify each region with SVMs.

Object Detection [Region Proposal]



Uijlings et al, “Selective Search for Object Recognition”, IJCV 2013

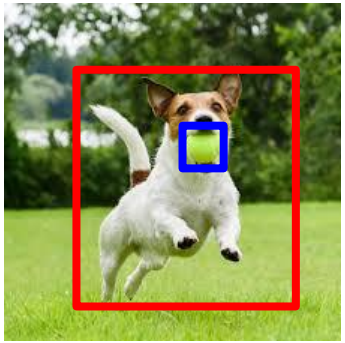
Fast R-CNN



- Use Selective Search to generate object proposals;
- Apply the CNN on the complete image and then used both **Region of Interest (RoI) Pooling** on the feature map;
- Use feed forward network for classification and regression;


fast
faster

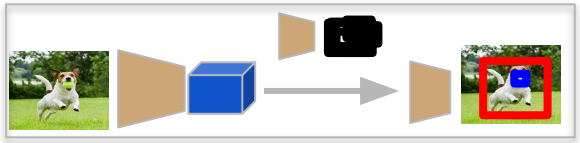
Faster R-CNN (Overall architecture)

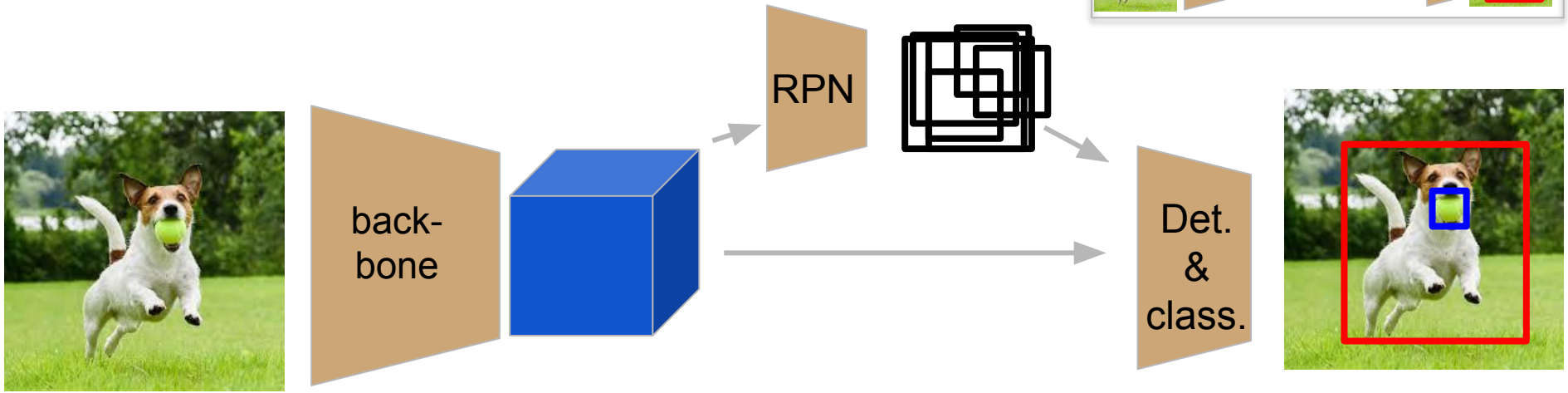


dog, score: 0.98
ball, score: 0.6

1506.01497

Faster R-CNN [Overall architecture]

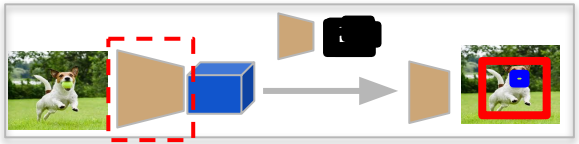


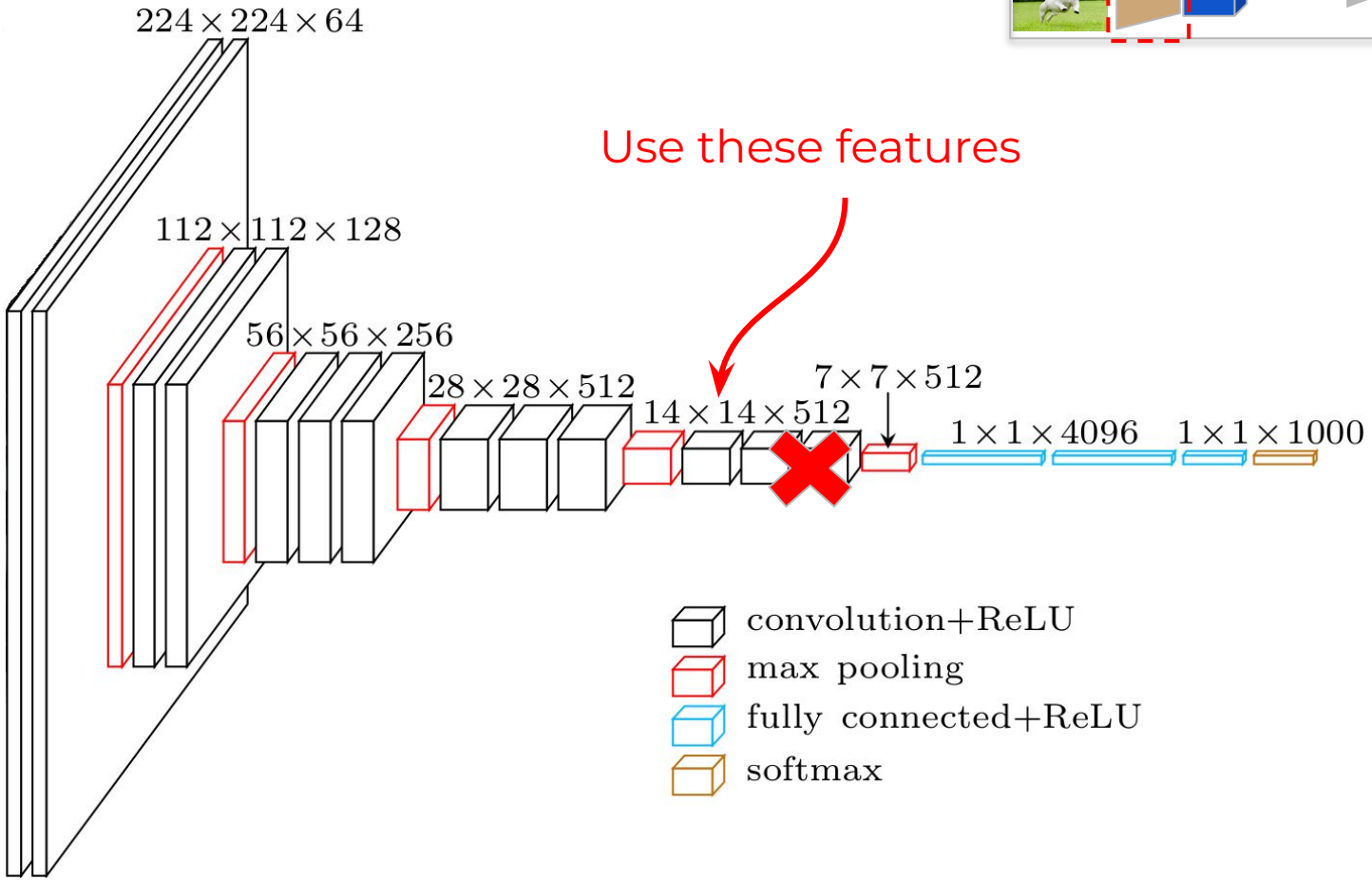


dog, score: 0.98
ball, score: 0.8

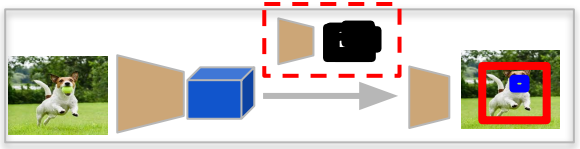
1506.01497

Faster R-CNN [Base Network]





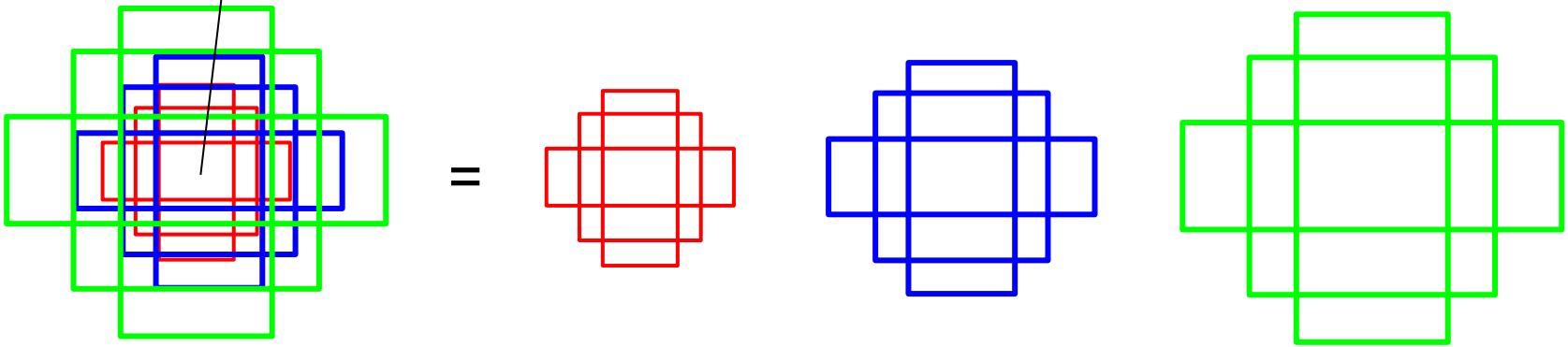
Faster R-CNN [Anchors]



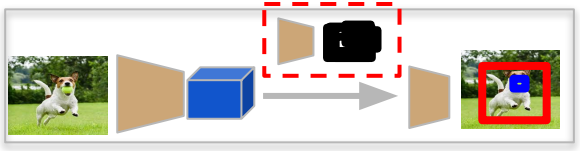
Anchors are fixed bounding boxes that are placed throughout the image with different sizes and ratios that are going to be used for reference when first predicting object locations.

- set of sizes (e.g. 64px, 128px, 256px)
- set of ratios between width and height of boxes (e.g. 0.5, 1, 1.5)

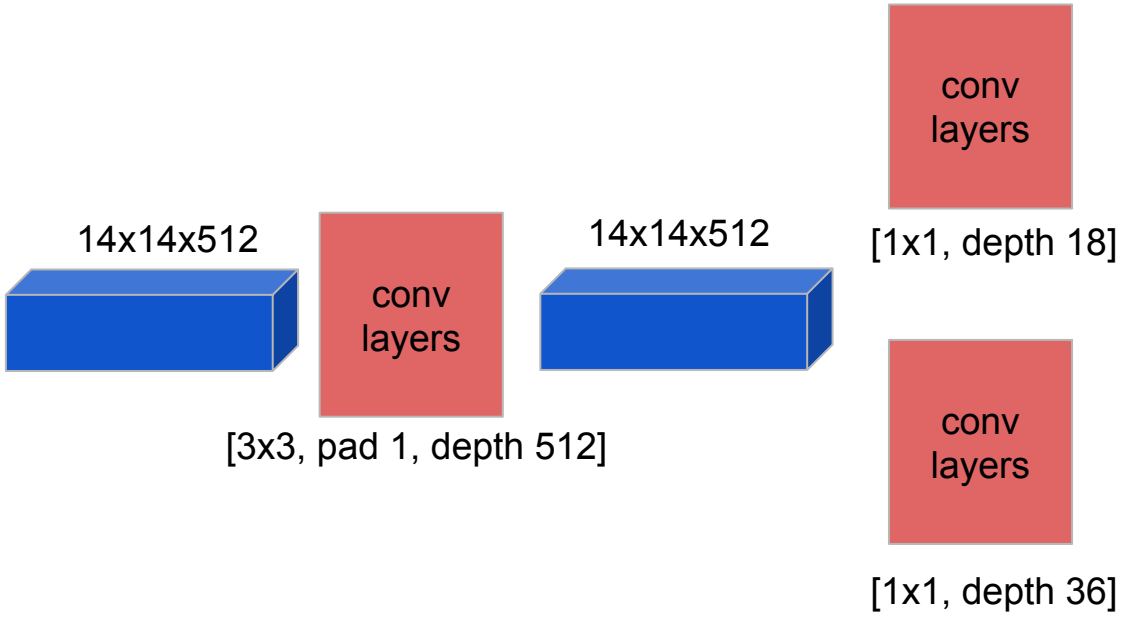
} = 9



Faster R-CNN [Region Proposal Network]



RPN takes all the reference boxes (anchors) and outputs a set of good proposals for objects.



18 = 2 * 9 = **2 * k**

14x14x18

Classification head
the score of it being background and the score of it being foreground (an actual object)

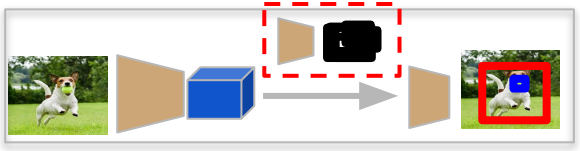
Regression head

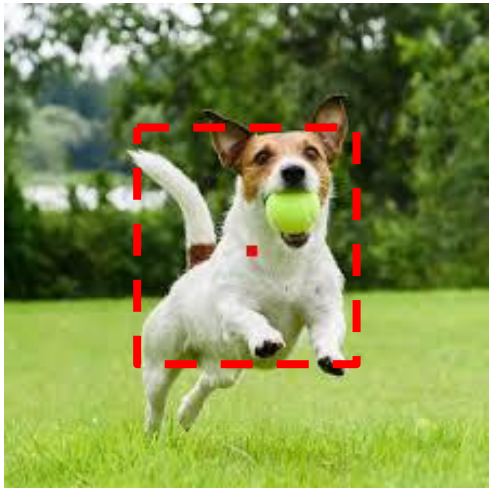
14x14x36

Δxcenter, Δycenter, Δw, Δh, which will be applied to the anchors to get the final proposals.

36 = 4 * 9 = **4 * k**

Faster R-CNN [Region Proposal Network]

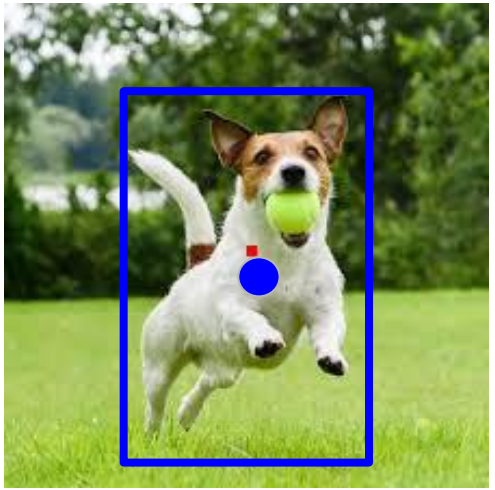




RPN output:

score (object) = 0.9
score (background) = 0.1

Δxcenter = 3
Δycenter = 10
Δw = 10
Δh = 40

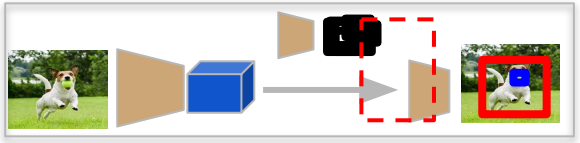


default anchor position and size:
Xa, Ya, Wa, Ha

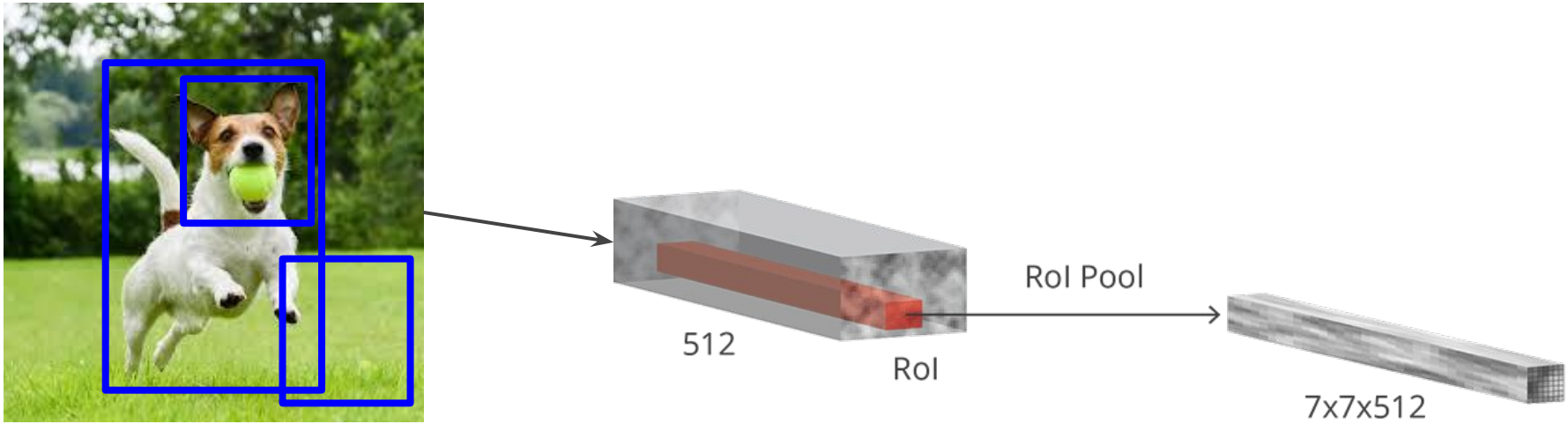
$$X = \Delta xcenter * Wa + Xa$$
$$Y = \Delta ycenter * Wa + Ya$$
$$W = Wa * exp(\Delta w)$$
$$H = Ha * exp(\Delta h)$$

proposal box position and size:
X, Y, W, H

Faster R-CNN [Region of Interest Pooling]

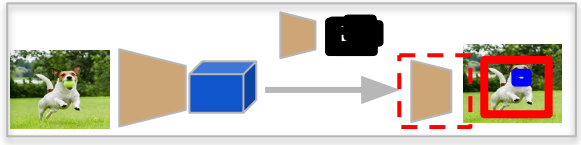


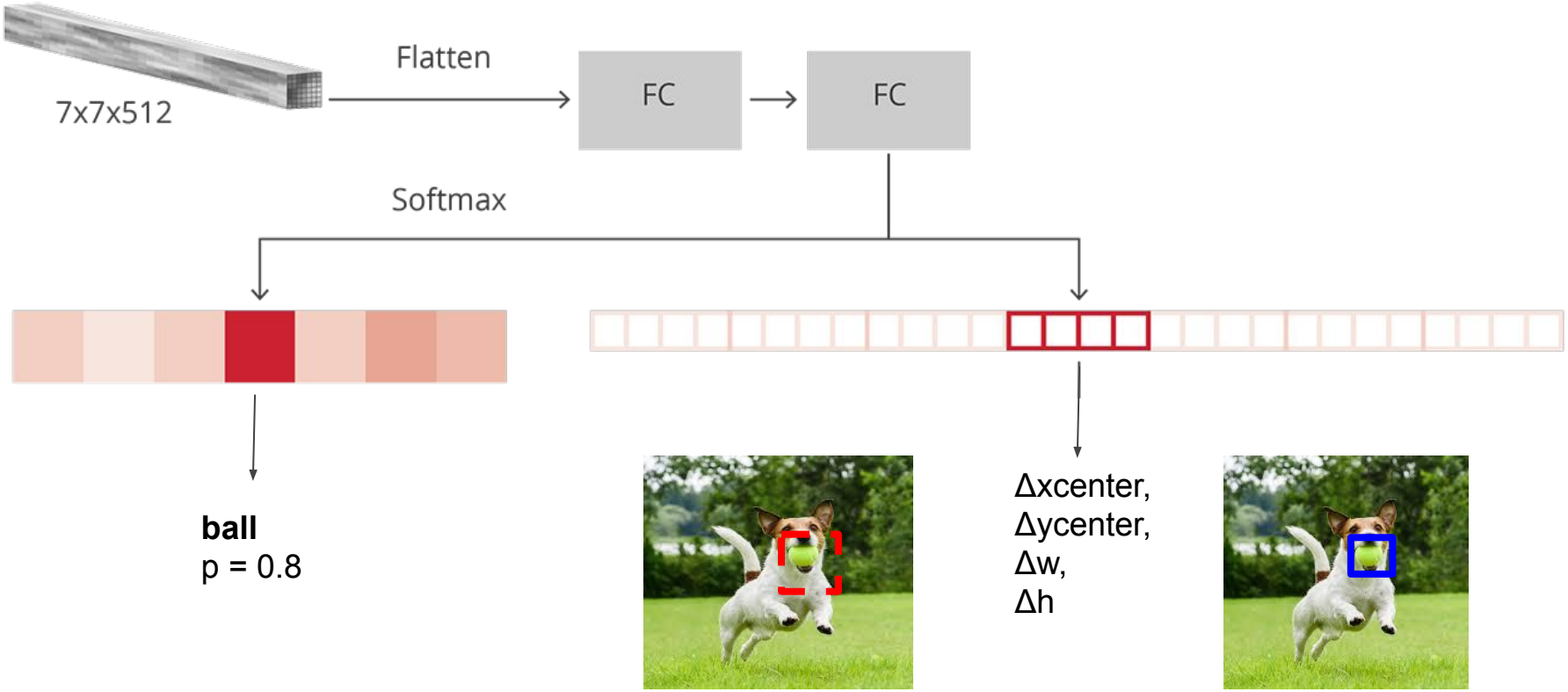
Fixed size feature maps are needed for the det.&class. part (R-CNN) in order to classify them into a fixed number of classes.



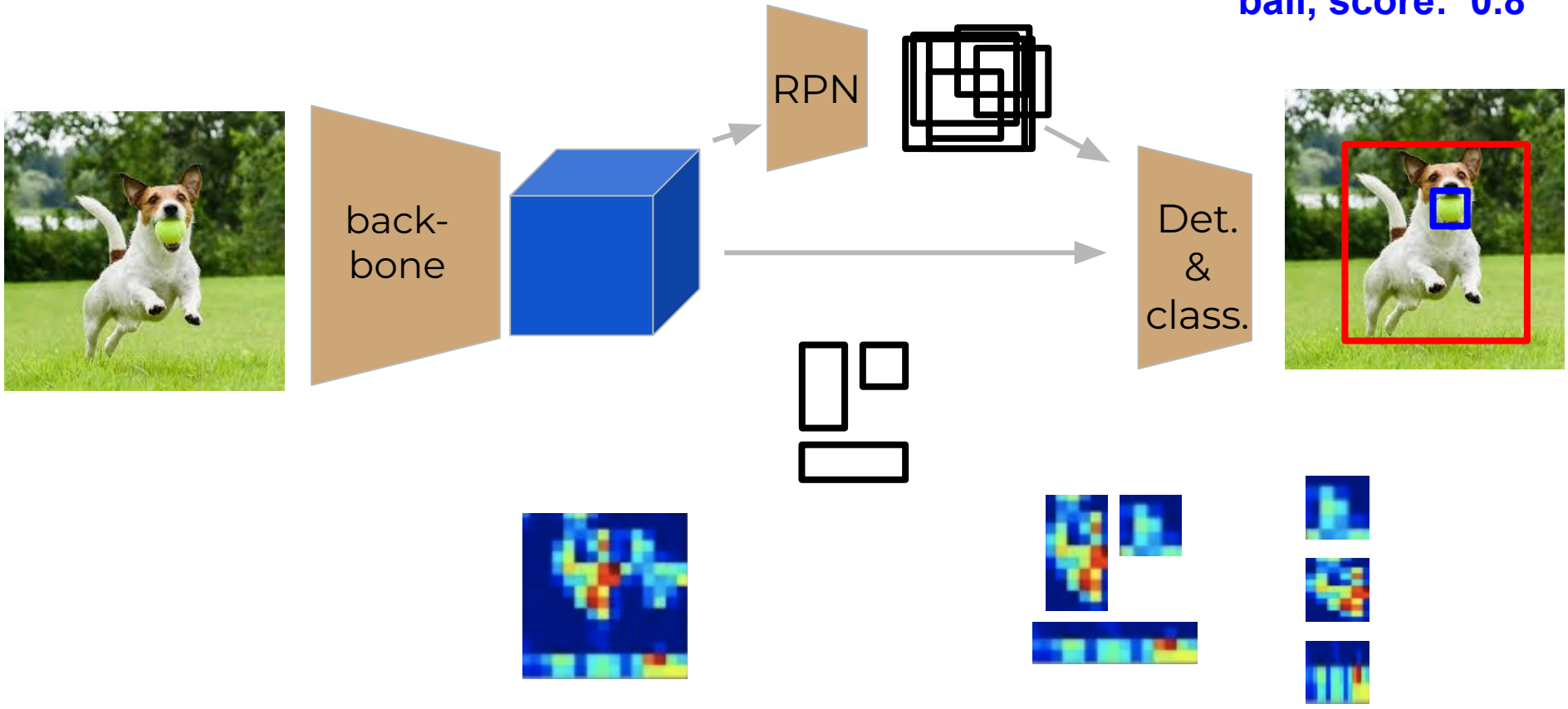
crop the convolutional features using each proposal and then resize to a fixed sized 14×14×D using interpolation (usually bilinear). After cropping, max pooling with a 2x2 kernel is used to get a final 7×7×D for each proposal.

Faster R-CNN [Region-based CNN]

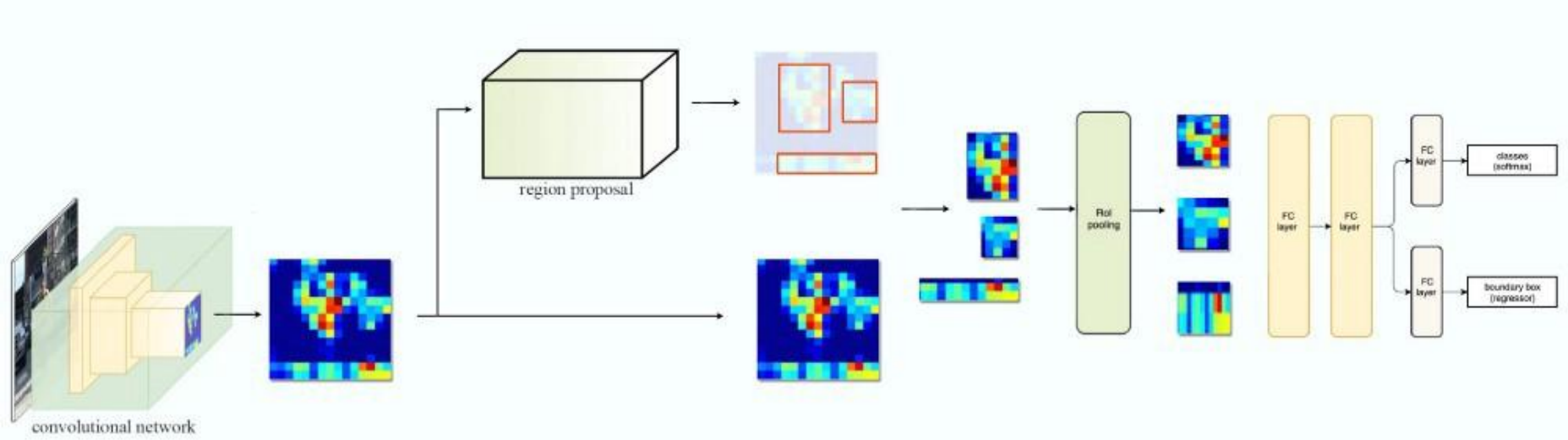




Faster R-CNN [Overall architecture]

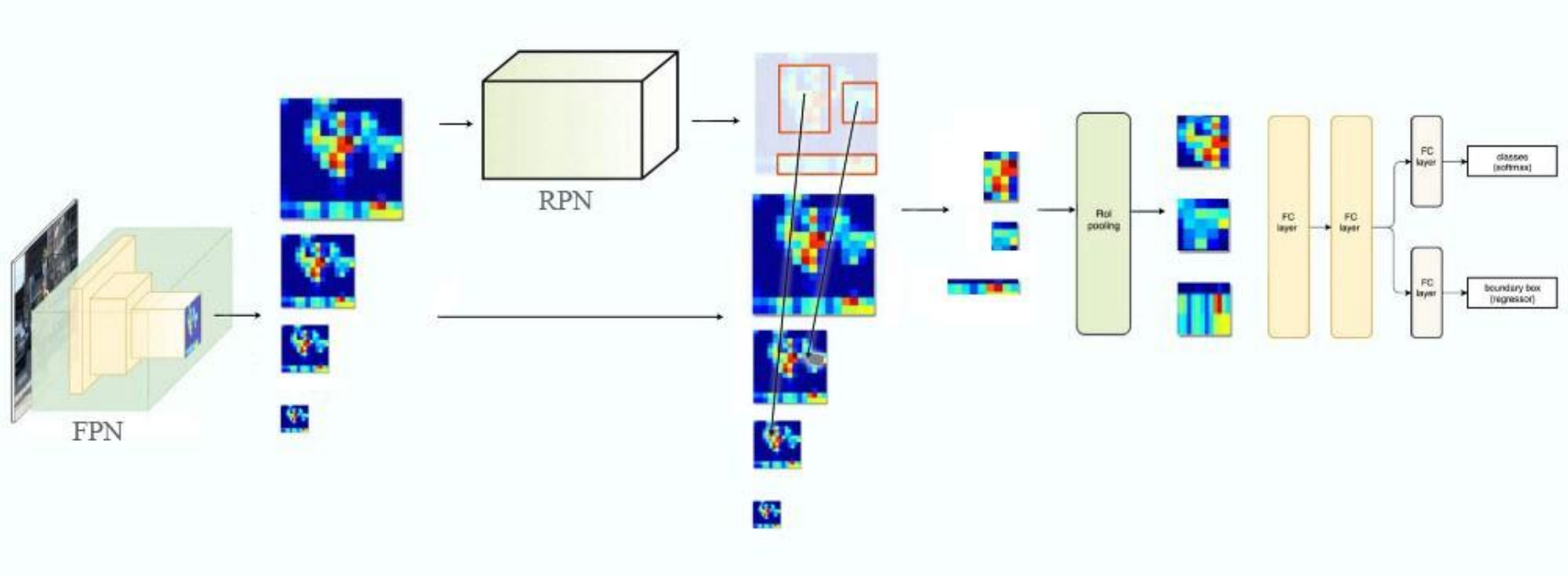


Faster RCNN (normal)



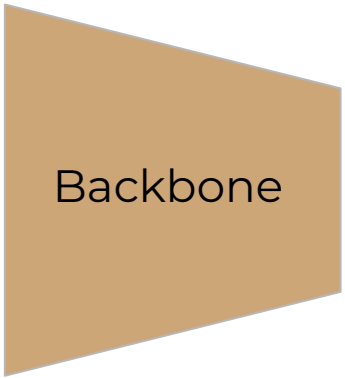
https://medium.com/@jonathan_hui/understanding-feature-pyramid-networks-for-object-detection-fpn-45b227b9106c

FPN for Faster RCNN



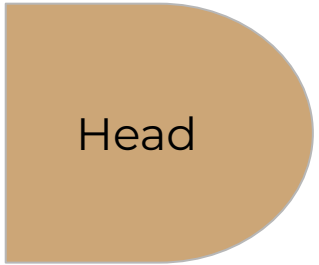
https://medium.com/@jonathan_hui/understanding-feature-pyramid-networks-for-object-detection-fpn-45b227b9106c

Faster RCNN



The backbone extracts vital features from the image

The neck refines these features, enhancing spatial and semantic information.

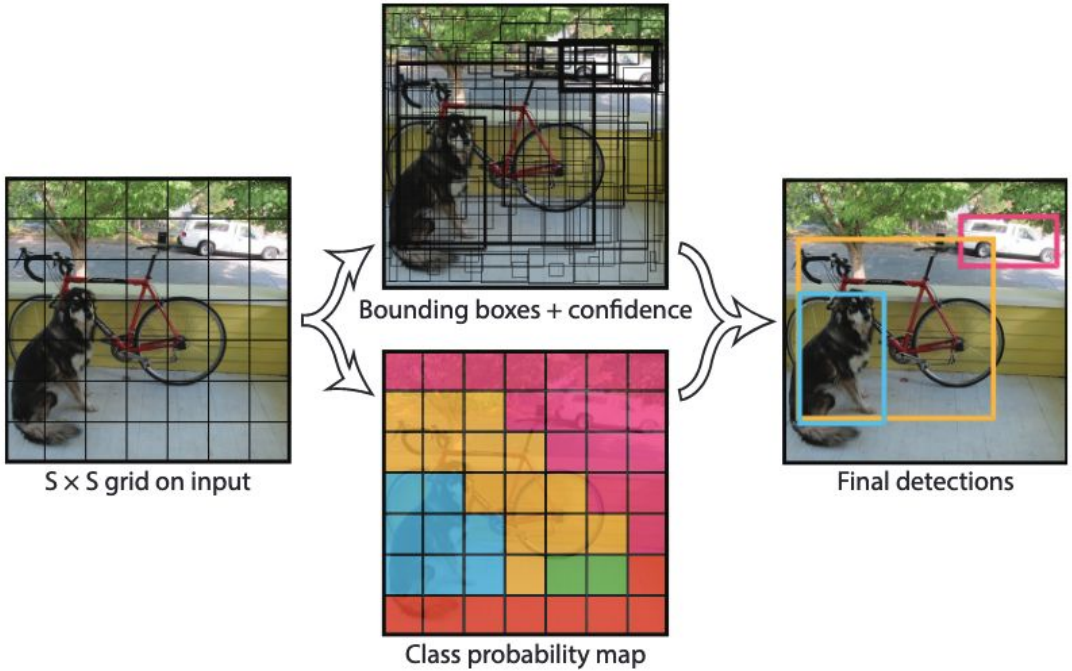


The head uses these refined features to make object detection predictions.

Content

- Intro to Object Detection
  - Datasets
  - Metrics
- Two-stage detectors [RCNN Family]
- **One-stage detectors**
  - YOLO
  - Single Shot Detector [SSD]
- Proposal-free detection [CenterNet]
- Transformer based detection [DETR]
- Open-World

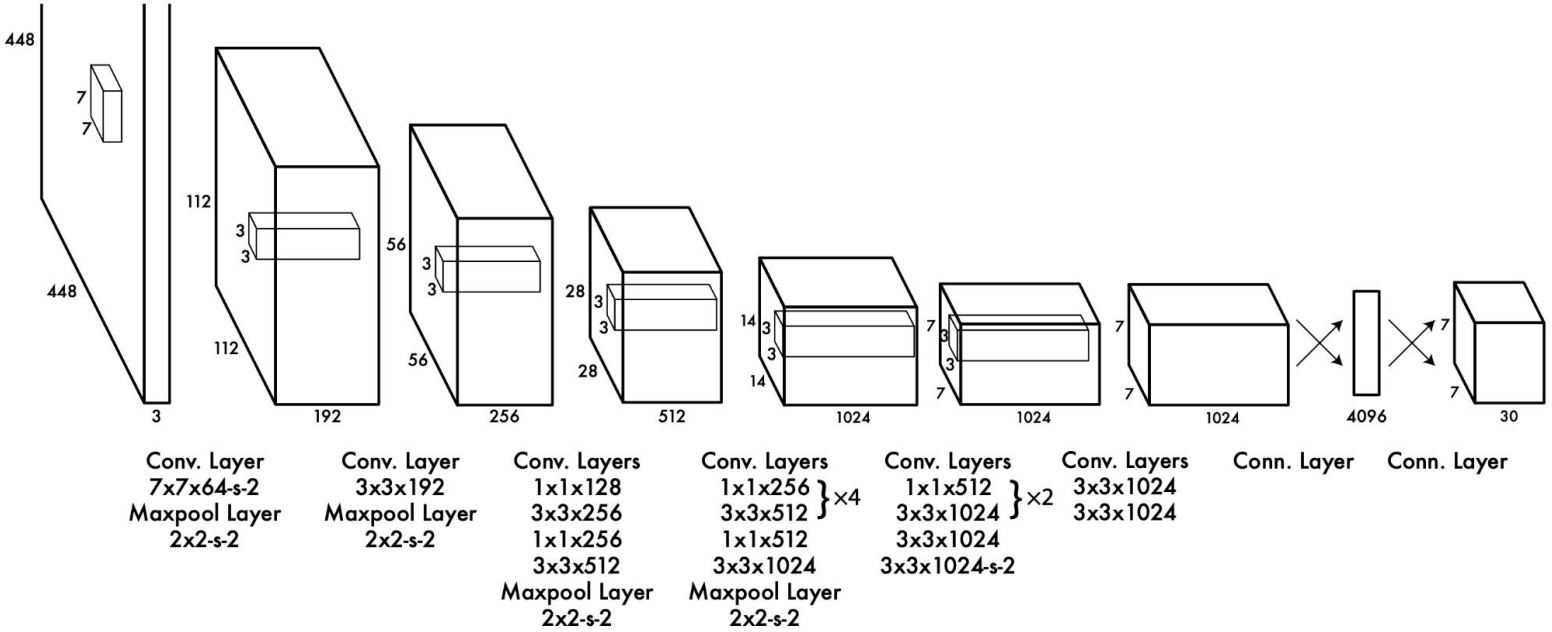
YOLO



YOLO models detection as a regression problem. It divides the image into an S ×S grid and for each grid cell predicts B bounding boxes, confidence for those boxes, and C class probabilities. These predictions are encoded as an S × S × (B * 5 + C) tensor

1506.02640

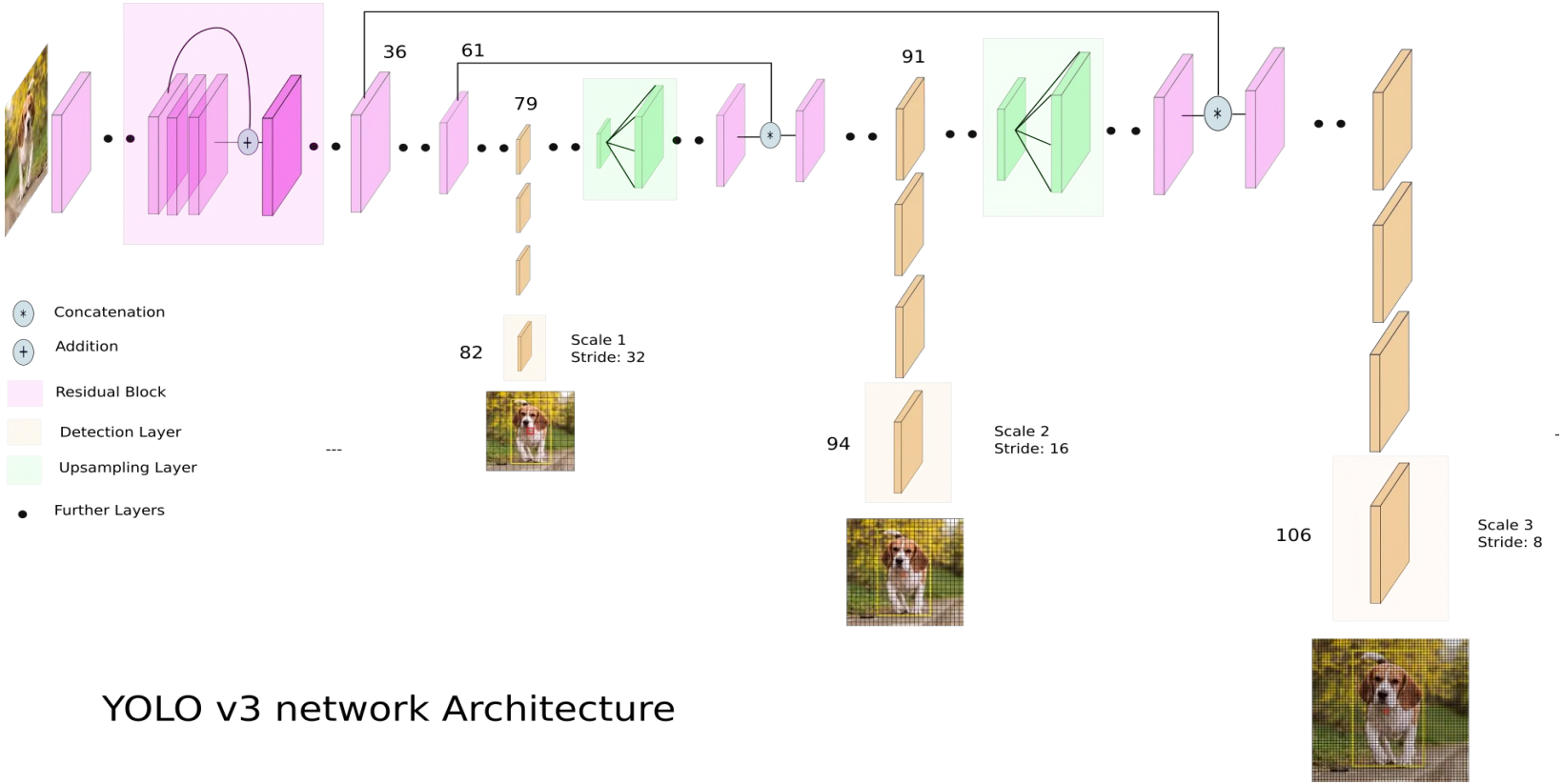
YOLO



Original paper used S = 7,B = 2
PASCAL VOC has 20 labelled classes so C = 20.
The final prediction is a 7 × 7 × 30 tensor.

1506.02640

YOLO



YOLO v3 network Architecture

YOLO

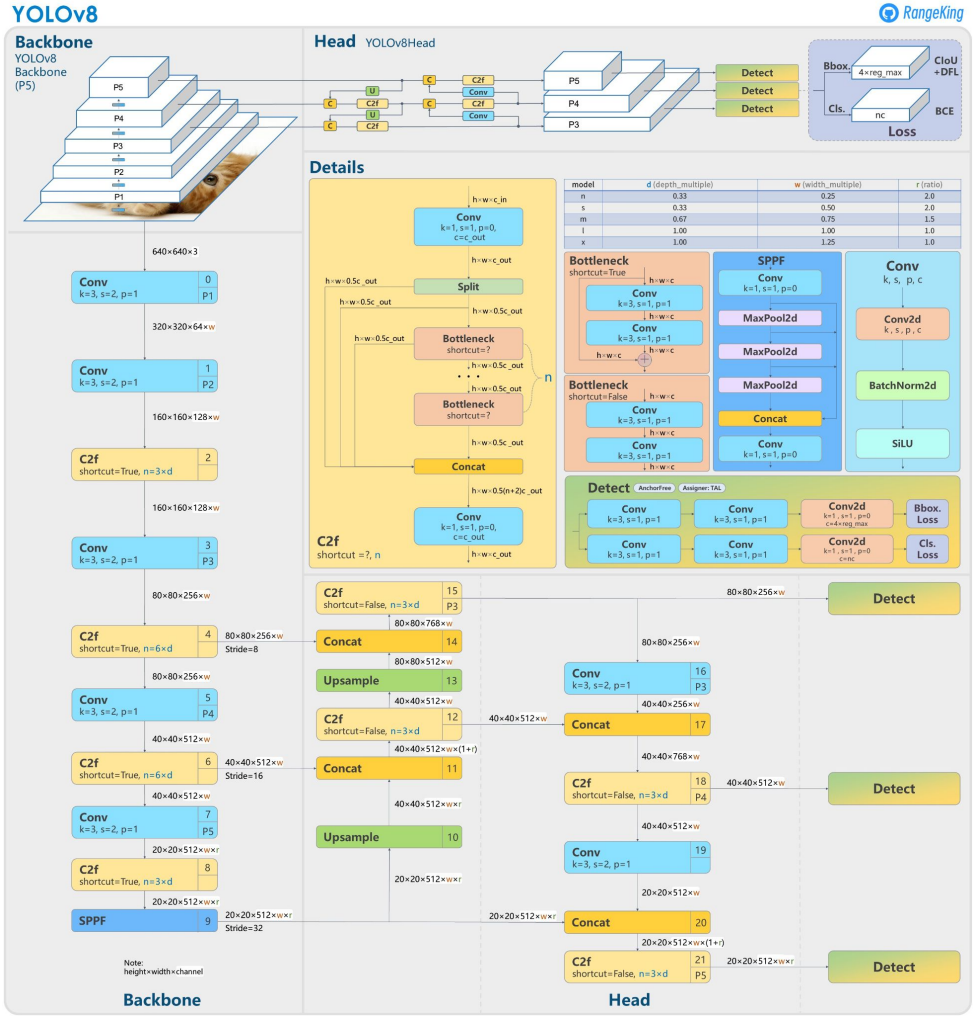
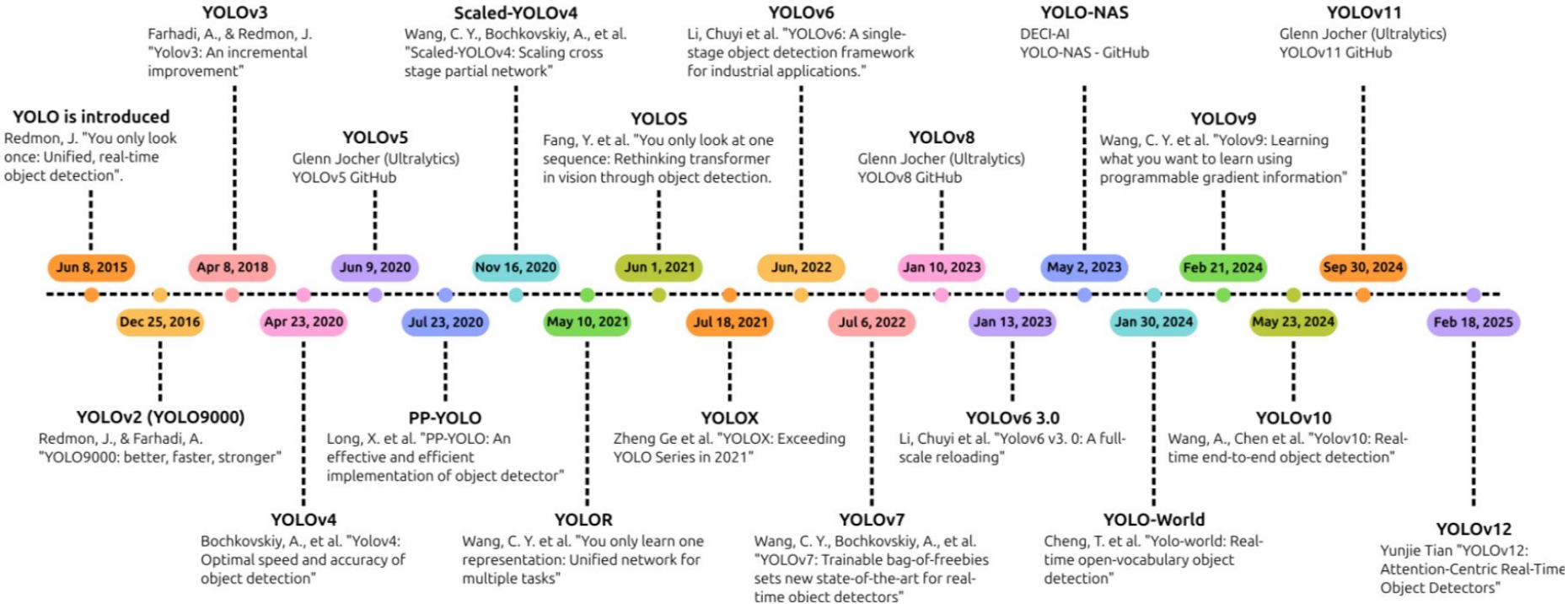


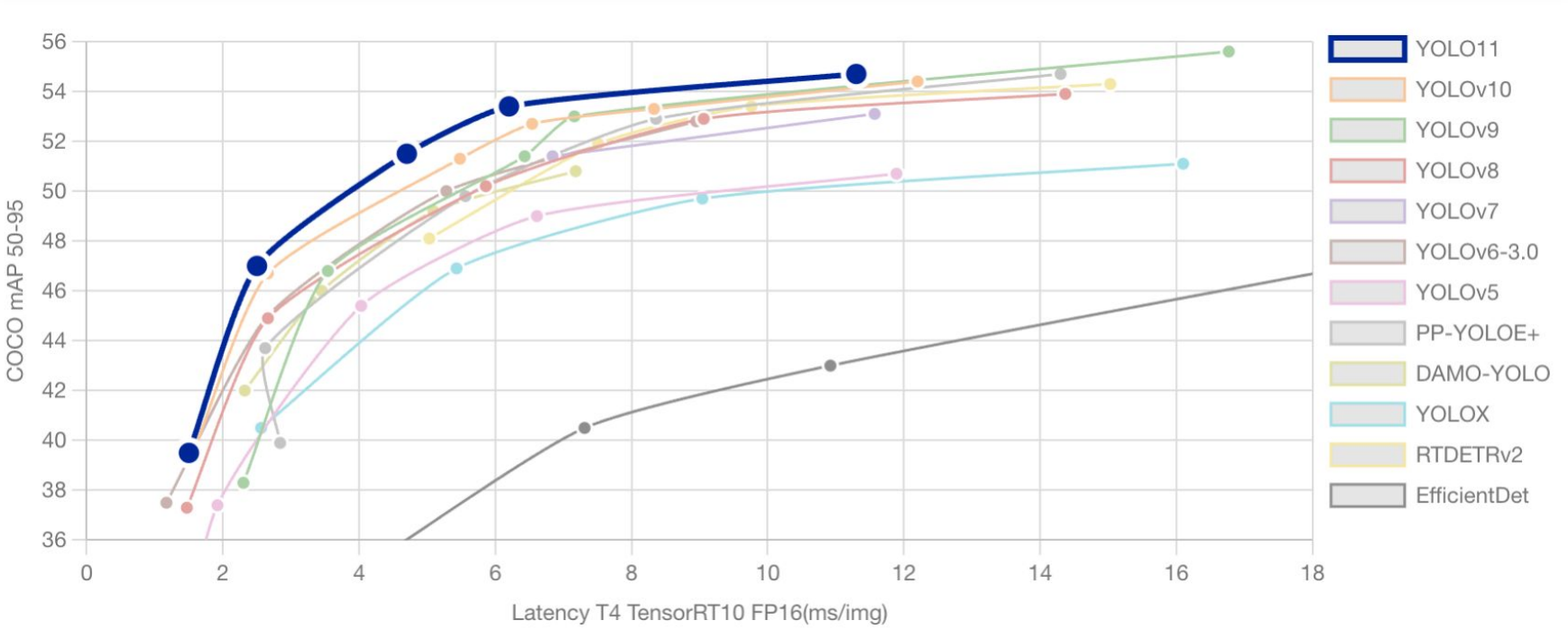
image link

YOLO



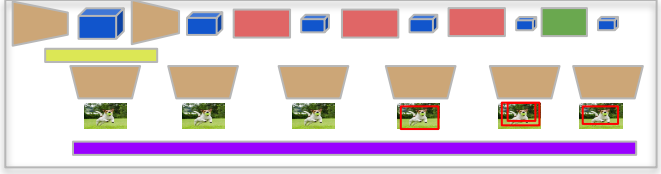
reviews: 2304.00501 2411.00201

YOLO



Ultralytics

SSD architecture



300x300x3

VGG up to conv4_3

38x38x512

VGG up to fc7

19x19x254

conv layers

10x10x512

conv layers

5x5x256

conv layers

3x3x256

avr. pool.

1x1x256

normalization

Det. & class.

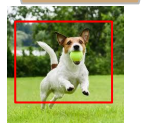
Det. & class.

Det. & class.

Det. & class.

Det. & class.

Det. & class.

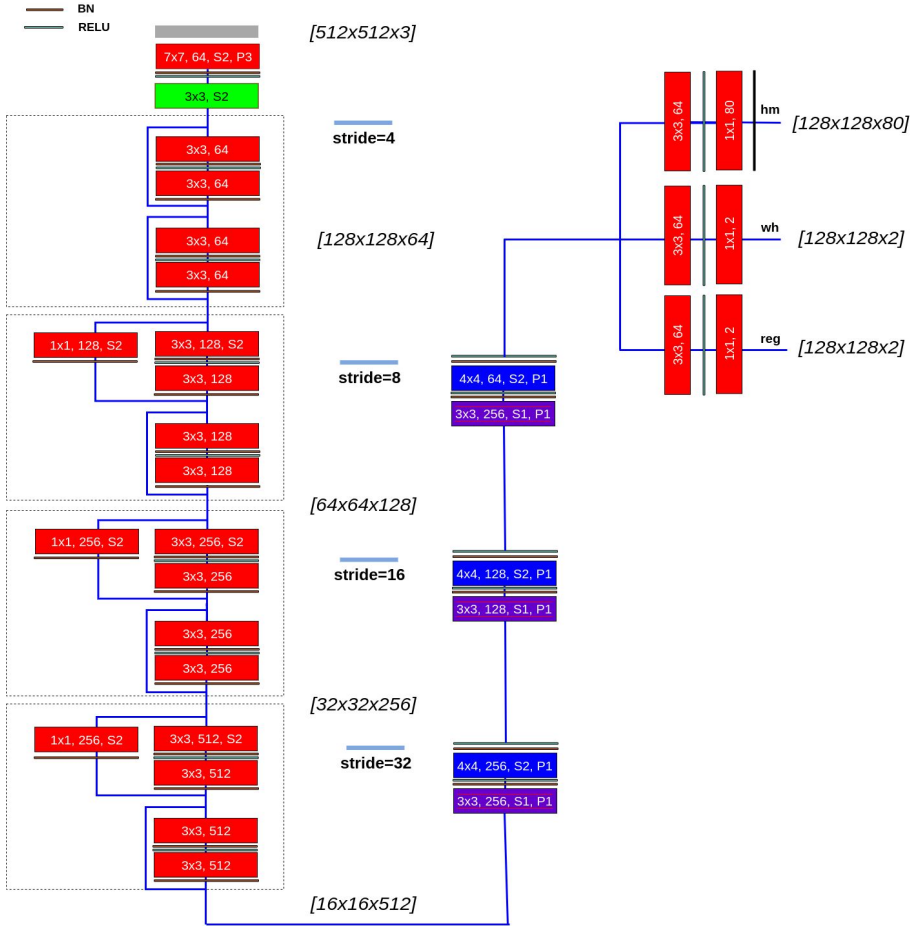


Fast NMS

Final detection

1512.02325

CenterNet [architecture]

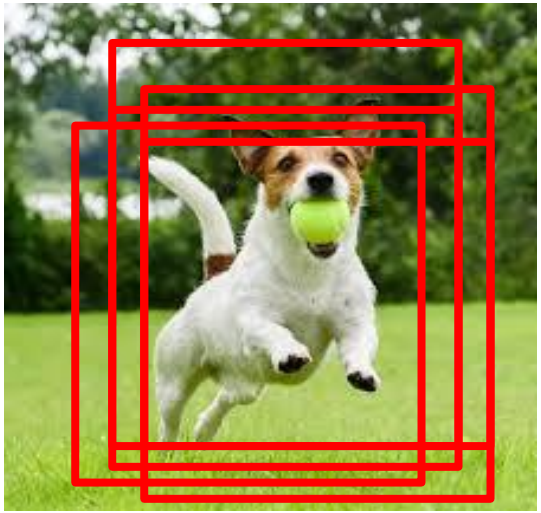


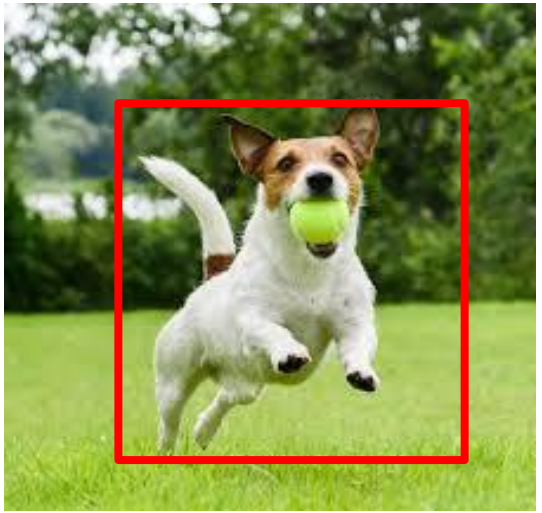
keypoint heatmap [C] local offset [2] object size [2]

$$(\hat{x}_i + \delta\hat{x}_i - \hat{w}_i/2,\ \ \hat{y}_i + \delta\hat{y}_i - \hat{h}_i/2,$$
$$\hat{x}_i + \delta\hat{x}_i + \hat{w}_i/2,\ \ \hat{y}_i + \delta\hat{y}_i + \hat{h}_i/2)$$

CenterNet (Objects as Points), *arXiv:* 1904.07850

Non-maximum Suppression (NMS)





Non-maximum Suppression (NMS)

Algorithm 1 Non-Maximum Suppression Algorithm

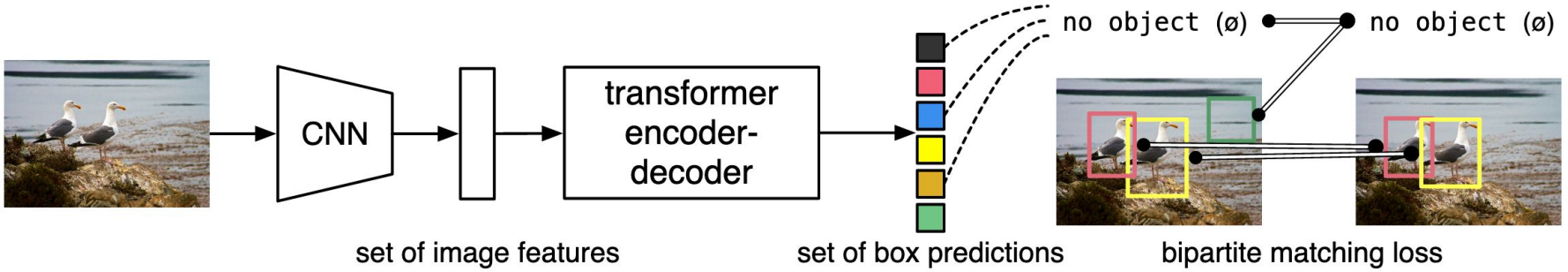
Require: Set of predicted bounding boxes B, confidence scores S, IoU threshold τ, confidence threshold T
Ensure: Set of filtered bounding boxes F

$F \leftarrow \emptyset$
Filter the boxes: $B \leftarrow \{b \in B \mid S(b) \geq T\}$
Sort the boxes B by their confidence scores in descending order
while $B \neq \emptyset$ **do**
    Select the box b with the highest confidence score
    Add b to the set of final boxes F: $F \leftarrow F \cup \{b\}$
    Remove b from the set of boxes B: $B \leftarrow B - \{b\}$
    for all remaining boxes r in B **do**
        Calculate the IoU between b and r: $iou \leftarrow IoU(b, r)$
        if $iou \geq \tau$ **then**
            Remove r from the set of boxes B: $B \leftarrow B - \{r\}$
        end if
    end for
end while

Content

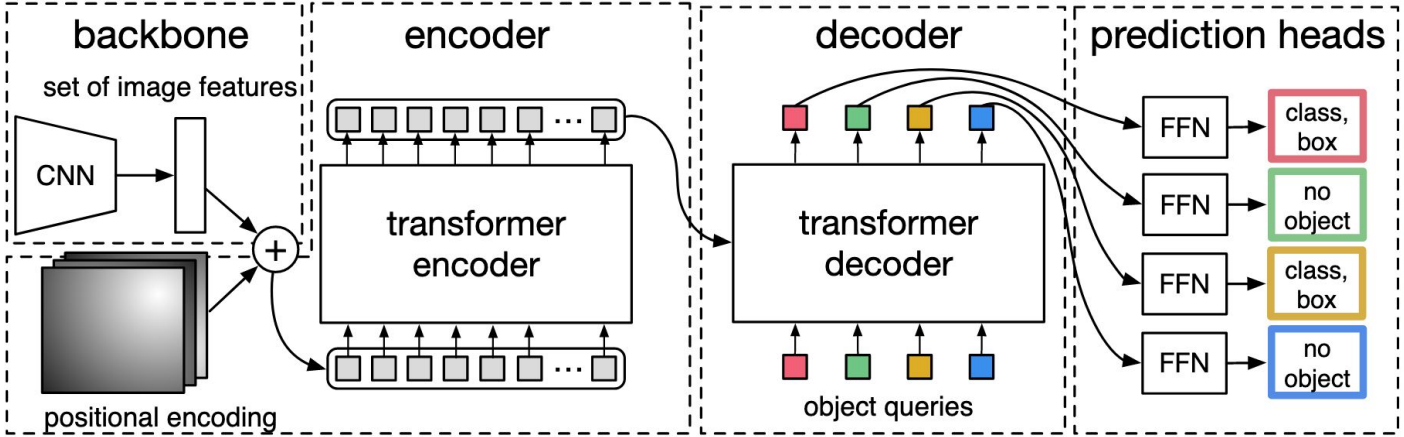
- Intro to Object Detection
  - Datasets
  - Metrics
- Two-stage detectors [RCNN Family]
- One-stage detectors
  - YOLO
  - Single Shot Detector [SSD]
- **Transformer based detection [DETR]**
- Open-World

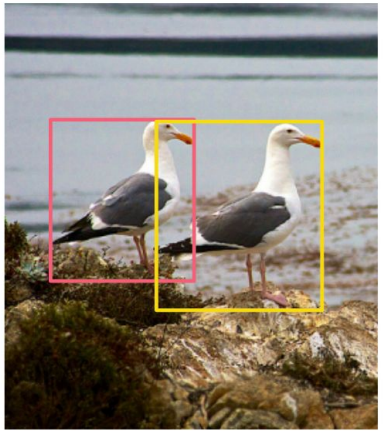
Detection with Transformers

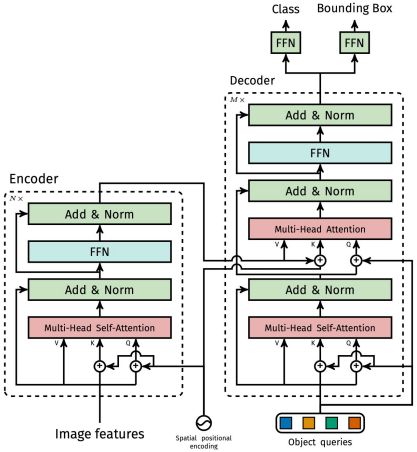


[2005.12872]

Detection with Transformers

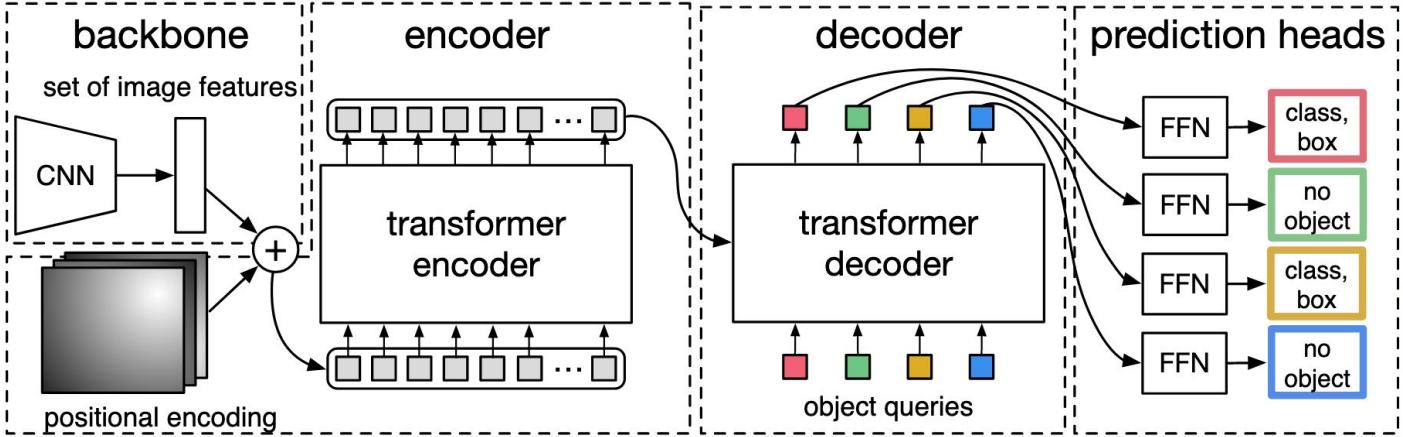


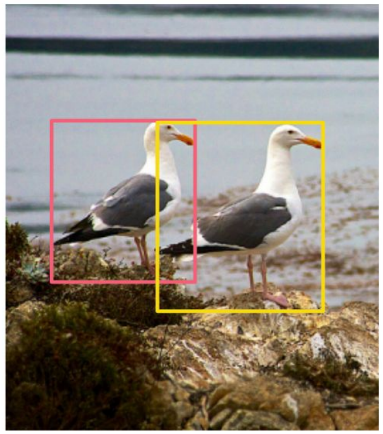


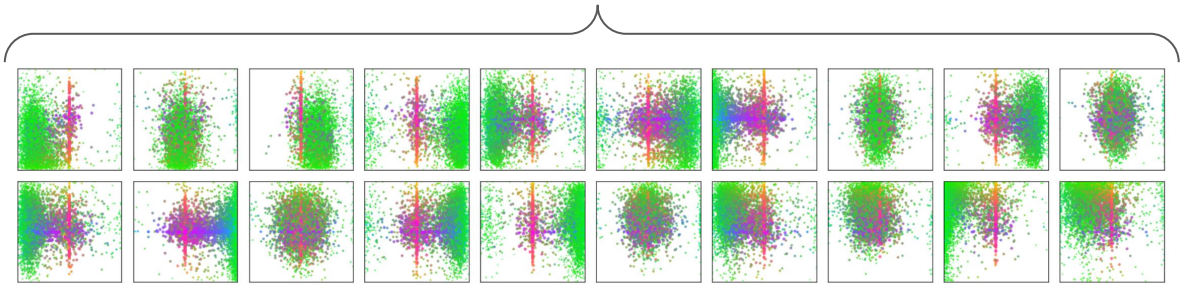


[2005.12872]

Detection with Transformers







[2005.12872]

Detection with Transformers

Model	GFLOPS/FPS	#params	AP	AP_{50}	AP_{75}	AP_S	AP_M	AP_L
Faster RCNN-DC5	320/16	166M	39.0	60.5	42.3	21.4	43.5	52.5
Faster RCNN-FPN	180/26	42M	40.2	61.0	43.8	24.2	43.5	52.0
Faster RCNN-R101-FPN	246/20	60M	42.0	62.5	45.9	25.2	45.6	54.6
Faster RCNN-DC5+	320/16	166M	41.1	61.4	44.3	22.9	45.9	55.0
Faster RCNN-FPN+	180/26	42M	42.0	62.1	45.5	26.6	45.4	53.4
Faster RCNN-R101-FPN+	246/20	60M	44.0	63.9	**47.8**	**27.2**	48.1	56.0
DETR	86/28	41M	42.0	62.4	44.2	20.5	45.8	61.1
DETR-DC5	187/12	41M	43.3	63.1	45.9	22.5	47.3	61.1
DETR-R101	152/20	60M	43.5	63.8	46.4	21.9	48.0	61.8
DETR-DC5-R101	253/10	60M	**44.9**	**64.7**	47.7	23.7	**49.5**	**62.3**

[2005.12872]

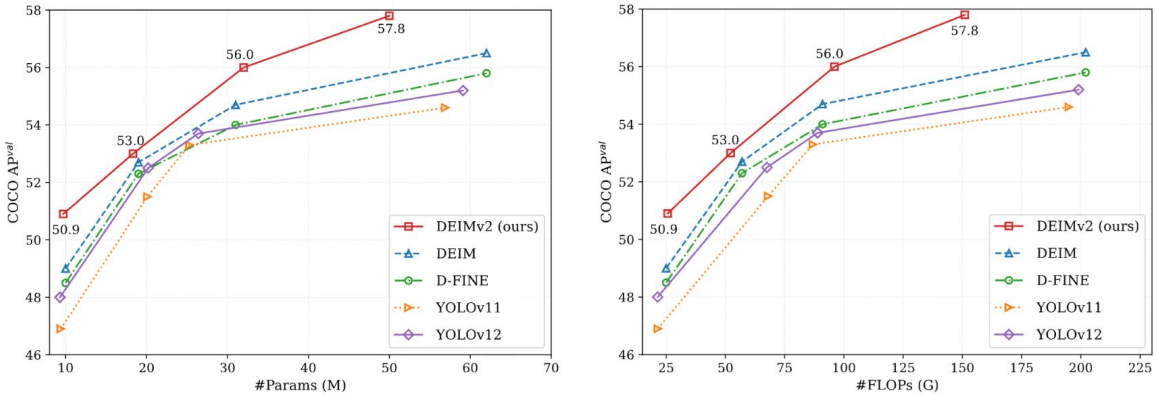
DETR vs YOLO

Real-Time Object Detection Meets DINOv3

Shihua Huang[1⋆], Yongjie Hou[1, 2⋆], Longfei Liu[1⋆], Xuanlong Yu[1], Xi Shen[1†]

[1] Intellindust AI Lab; [2]Xiamen University
⋆ Equal Contribution; † Corresponding author.

Project Page: https://intellindust-ai-lab.github.io/projects/DEIMv2
Code & Weights: https://github.com/Intellindust-AI-Lab/DEIMv2



(a) **COCO performance v.s. Number of Parameters.** (b) **COCO performance v.s. FLOPs.**

Figure 1. Compared with state-of-the-art real-time object detectors on COCO [12], all variants of our proposed DEIMv2 (S, M, L, X) achieve superior performance in terms of average precision (AP) while maintaining similar parameters and less computational cost.

[2509.20787]

Content

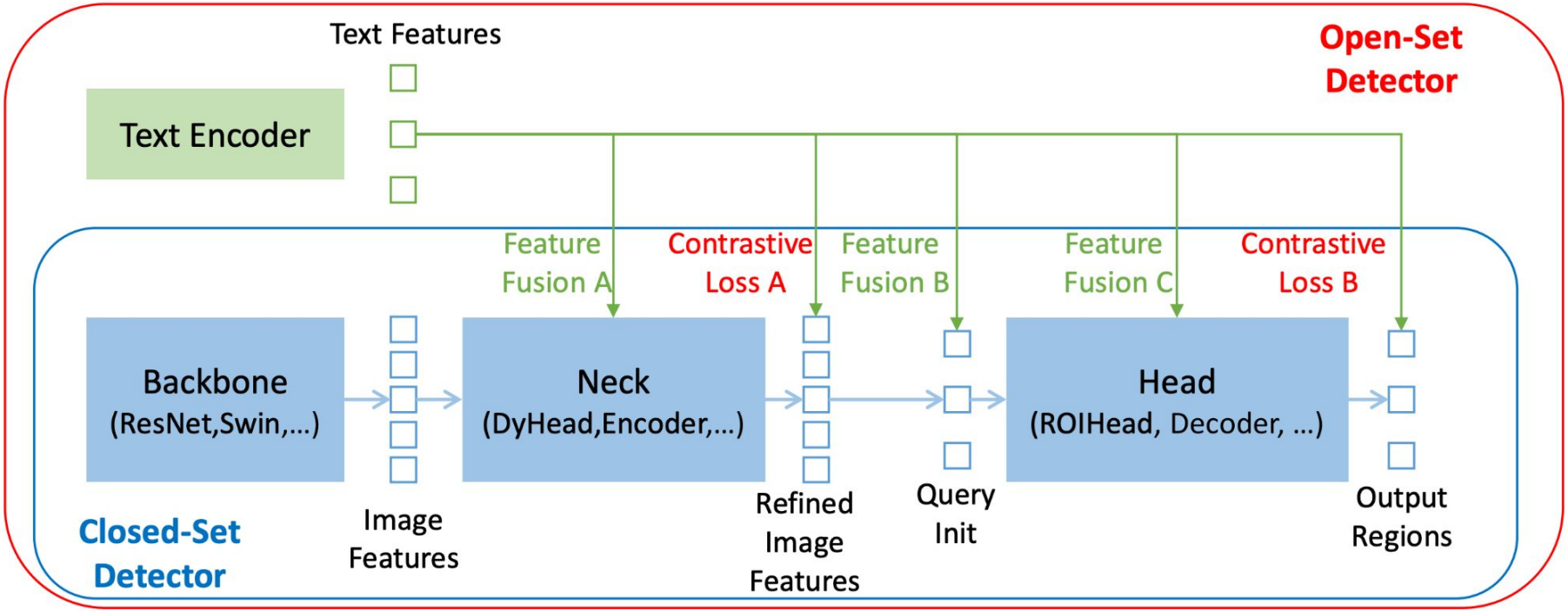
- Intro to Object Detection
  - Datasets
  - Metrics
- Two-stage detectors [RCNN Family]
- One-stage detectors
  - YOLO
  - Single Shot Detector [SSD]
- Transformer based detection [DETR]
- **Open-World**

Open-World

Detect: dog, cake

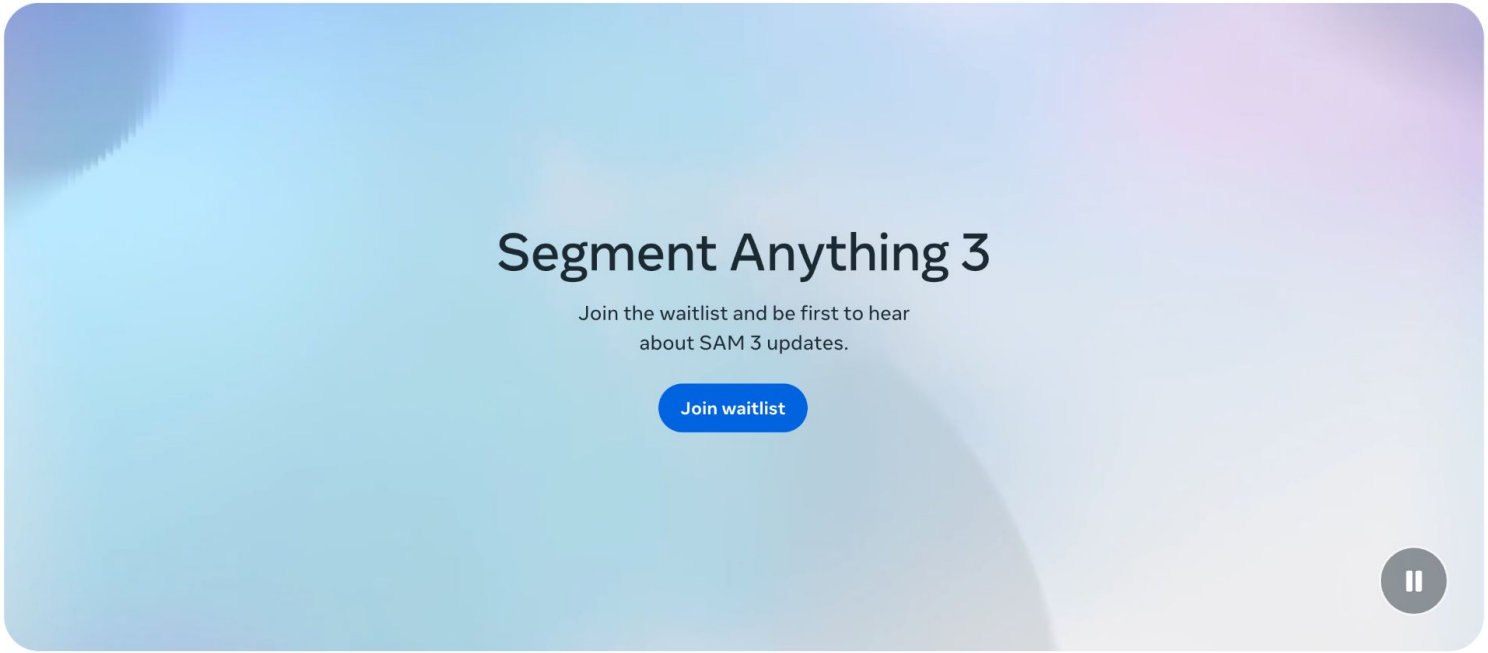
2303.05499

Open-World



2303.05499

Open-World



https://www.linkedin.com/posts/arun-ponnusamy_meta-computervision-sam3-activity-7340777786929836032-nwOc/

Goals of the Course

- Get big picture of the deep learning for CV
- Working knowledge of essential elements/blocks of **Convolutional Neural Networks [CNNs]**
- Working knowledge of essential elements/blocks of **Transformers** and their use in CV
- Get flavor of CV-related **Foundational models**
- Get deeper with one particular problem (**Object Detection**)

EPAM is hiring data scientists!